

INFORME FINAL: CLASIFICACIÓN AUTOMÁTICA DE TEXTO

NICOLAS HERRERA MARULANDA
202182551

Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación
Programa Académico de Ingeniería de Sistemas
Cali, Julio de 2025

Resumen

Este taller presenta el desarrollo de un sistema de clasificación automática de reseñas de negocios utilizando el conjunto de datos Yelp Review Dataset. El objetivo principal fue predecir la calificación otorgada (de 1 a 5 estrellas) a partir del texto de la reseña empleando diversas arquitecturas de redes neuronales. En la primera fase se implementó un Perceptrón Multicapa como línea base. Posteriormente, se exploraron arquitecturas avanzadas incluyendo RNN, LSTM, y modelos basados en Transformers. Todo el desarrollo se realizó individualmente, utilizando Python junto con librerías de aprendizaje automático como Keras y Tensorflow

Tabla de contenidos

Introducción....4
Contenido....5
- Primera Entrega....5
  - Preprocesamiento....5
  - MLP....5
    - Modelo....5
    - Resultados....7
    - Análisis....7
- Segunda Entrega....9
  - Experimentos....9
    - Clasificación Binaria....9
    - Clasificación Multiclase....9
  - Preprocesamiento....9
  - Desbalanceo....10
  - RNN....11
    - Modelo....11
    - Resultados....12
      - Binario....12
      - Multiclase....15
      - Mejores Modelos....18
        - Binario Balanceado....18
        - Multiclase Balanceado....18
    - Análisis....19
      - Binario....19
      - Multiclase....20
      - General....21
  - LSTM....21
    - Modelo....21
    - Resultados....23
      - Binario....23
      - Multiclase....25
      - Mejores Modelos....28
        - Binario Balanceado....28
        - Multiclase Balanceado....29
    - Análisis....29
      - Binario....29
      - Multiclase....30
      - General....31
  - Transformer....31
    - Configuraciones:....32
    - Modelo....33
    - Resultados....33

Binario..... 33
Multiclase.....34
Análisis..... 36
Binario..... 36
Multiclase.....36
General.....37
Conclusiones.....39
Bibliografía..... 41

Introducción

La predicción automática de calificaciones en reseñas escritas es un desafío en el procesamiento de lenguaje natural, ya que permite a las plataformas identificar rápidamente la percepción de los usuarios sobre distintos negocios, productos y servicios. El presente taller aborda este problema utilizando el Yelp Review Dataset, que contiene 10,000 reseñas, cada una asociada a una calificación de 1 a 5 estrellas.

El objetivo del taller es comparar el desempeño de distintas arquitecturas de redes neuronales en la tarea de clasificación de texto multiclase. Para ello, se realizaron dos entregas principales: la primera consistió en el entrenamiento y evaluación de un modelo de Perceptrón Multicapa, mientras que la segunda amplió el análisis a modelos secuenciales y de atención, como RNN, LSTMy Transformers.

Se emplearon herramientas y librerías de programación (Python, Keras, Tensorflow, PyTorch) para facilitar la experimentación y análisis. El informe presenta el proceso de diseño, experimentación y comparación de los modelos, así como los aprendizajes y conclusiones obtenidas, contribuyendo a la comprensión del potencial y limitaciones de las arquitecturas neuronales en problemas de clasificación de texto.

Contenido

Primera Entrega

Preprocesamiento

Antes de alimentar el texto al modelo, se aplicaron varias etapas de preprocesamiento utilizando herramientas de Keras, con el objetivo de transformar el texto en un formato numérico adecuado para el entrenamiento con los diferentes modelos.

- **Estandarización del texto:** Se utilizó la función de estandarización integrada en la capa TextVectorization de Keras. Esta transformación convierte todo el texto a minúsculas y elimina los signos de puntuación, lo que reduce la dimensionalidad del vocabulario y mejora la consistencia del procesamiento semántico.
- **Tokenización basada en índices:** Cada palabra del vocabulario fue asignada a un índice entero único. Esta técnica convierte cada reseña en una secuencia de enteros, donde cada número representa un token distinto.
- **Vectorización y embeddings:** Las secuencias enteras fueron procesadas por la capa TextVectorization, que transforma texto en tensores de índices, y luego pasadas a una capa de embeddings de dimensión entre 16 y 32. Esta capa permite representar cada palabra como un vector denso y continuo que captura información semántica.
- **Límite de vocabulario**: Se fijó en 20,000 palabras para restringir el espacio semántico.
- **Longitud máxima de secuencia**: Se estableció en 348 tokens, correspondiente al percentil 95 de la distribución de longitudes en el conjunto de datos. Esto asegura que el 95% de las reseñas se mantengan completas o casi completas, minimizando la pérdida de información por truncamiento.
- **División del conjunto de datos**:
    - 70% entrenamiento
    - 10% validación
    - 20% prueba (test)

MLP

Modelo

El perceptrón multicapa (MLP) es una red neuronal feedforward compuesta por una capa de entrada, una o más capas ocultas densamente conectadas y una capa de salida.

El modelo es adecuado para tareas de clasificación supervisada, pero presenta limitaciones al trabajar con texto, ya que no puede capturar relaciones contextuales o secuenciales entre palabras, lo cual es clave en el procesamiento del lenguaje natural.

El modelo MLP implementado consistió en:

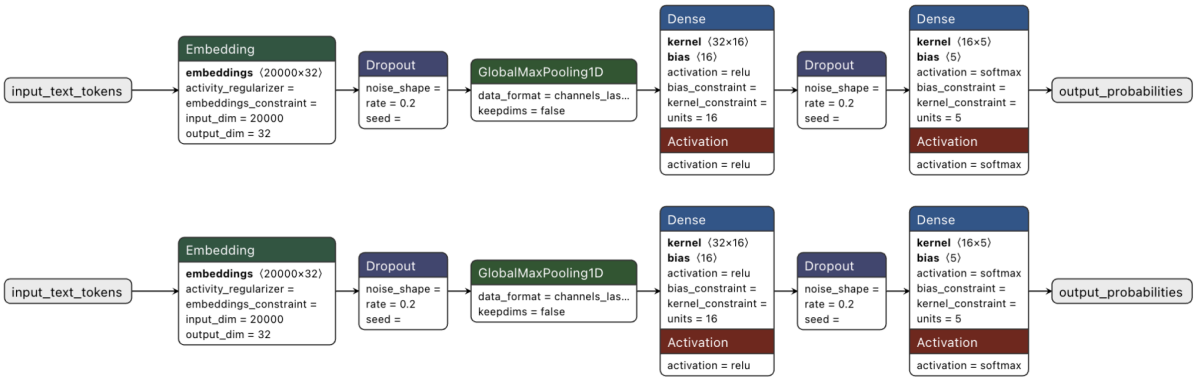
- **Capa de vectorización de texto**

Esta capa convierte las cadenas de texto (reseñas) en secuencias de enteros, donde cada entero representa un token del vocabulario.

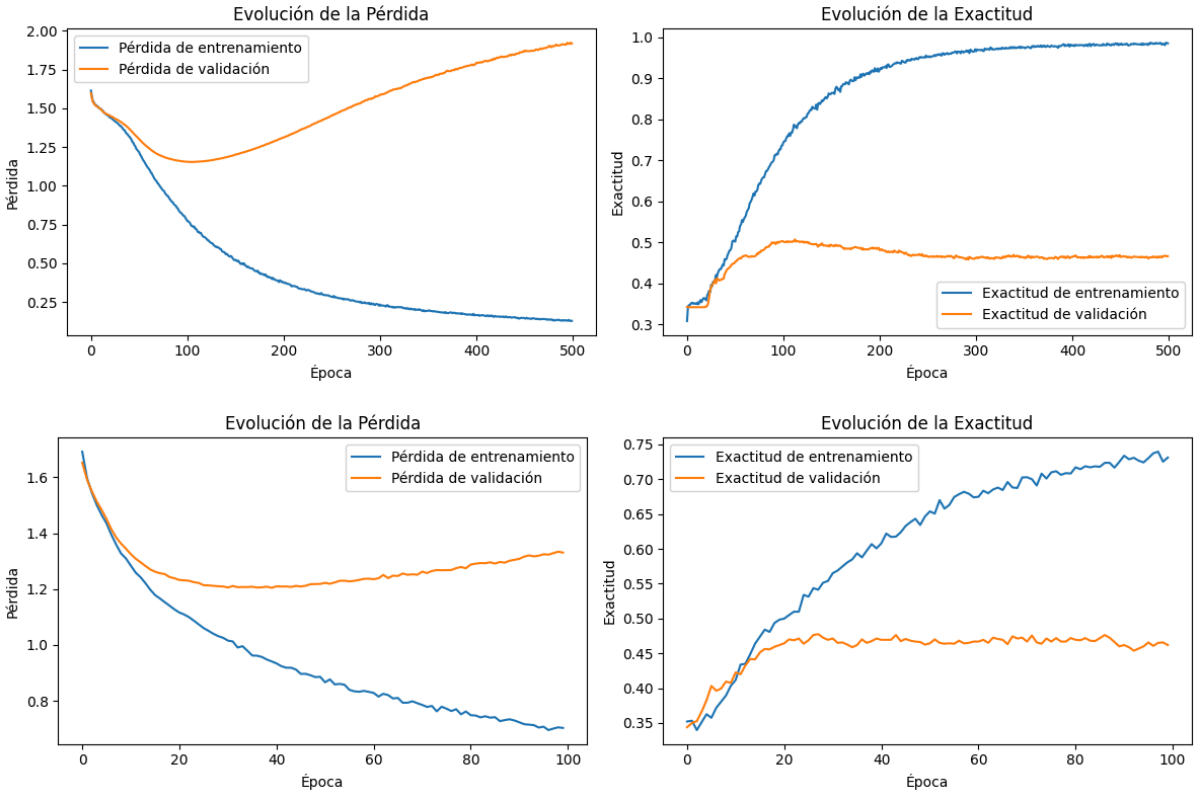
- **Capa Embedding**
  Transforma cada índice de palabra en un vector denso de dimensión fija.
  Permite capturar relaciones semánticas entre palabras. Por ejemplo, palabras similares en contexto tendrán vectores cercanos.
- **Capa de GlobalAveragePooling**
  Reduce la secuencia de vectores de embeddings a un único vector al promediar todas las posiciones.
  Sirve como un resumen del contenido general del texto. Aunque pierde el orden de las palabras, ofrece una forma eficiente de condensar la información semántica antes de pasarla a las capas densas.
- **Dos capas densas con activaciones ReLU**
  La función ReLU (Rectified Linear Unit) es computacionalmente eficiente y ayuda a mitigar el problema del desvanecimiento del gradiente, facilitando el aprendizaje en capas profundas.
- **Capa de salida con activación Softmax para clasificación multiclase (5 clases)**
  Porque es una tarea de clasificación multiclase, donde cada entrada debe ser clasificada en exactamente una de las cinco categorías. Softmax garantiza que las salidas sumen 1 y puedan interpretarse como probabilidades.

Configuraciones:

- **Función de pérdida:**
  Categorical Crossentropy, adecuada para tareas de clasificación multiclase con clases mutuamente excluyentes, como en el caso de reseñas etiquetadas de 1 a 5 estrellas.
- **Optimizador utilizado:**
  Adam, elegido por su capacidad de adaptación automática de la tasa de aprendizaje y su buen desempeño en problemas de texto.
- **Regularización:**
  Se aplicaron técnicas de *Dropout* y penalización *L2* en las capas densas para mitigar el sobreajuste observado durante las primeras pruebas. Estas técnicas ayudaron a mejorar la generalización, aunque sus efectos sobre la precisión final variaron según la configuración.
- **Tamaño de batch:**
  Se evaluaron valores de 32 y 128.
- **Épocas:**
  Se exploraron dos configuraciones principales: 100 y 500 épocas.



Resultados



Análisis

El desempeño del modelo MLP estuvo marcado por una fuerte tendencia al sobreajuste y una limitada capacidad de generalización. A continuación se detallan los hallazgos más relevantes:

- **Generalización limitada:**
  Aunque el modelo logró ajustar bien los datos de entrenamiento, su rendimiento en el conjunto de validación fue significativamente inferior. Esto se debe a que los perceptrones multicapa (MLP) no son capaces de capturar relaciones contextuales o dependencias entre palabras en secuencias de texto. Como resultado, el modelo tiende a memorizar ejemplos específicos en lugar de aprender patrones generalizables.

- **Ajuste de parámetros:**
  Incrementar el número de neuronas en las capas ocultas y la dimensión del espacio de embedding mejoró el rendimiento en entrenamiento, pero a costa de una mayor tasa de error en validación. Esto indica que el modelo se volvió más complejo y sobreajustó los datos, lo que resultó en una disminución de la precisión en datos no vistos.
- **Regularización y Dropout:**
  La incorporación de regularización L2 y la aplicación de dropout (junto con una reducción en el número de neuronas) ayudaron a disminuir el sobreajuste y lograron una ligera mejora en la precisión de validación. Sin embargo, estas medidas también redujeron el rendimiento en entrenamiento, lo que sugiere un balance delicado entre capacidad de aprendizaje y generalización.
- **Aumento de profundidad:**
  Al incrementar el número de capas ocultas, no se observaron mejoras significativas en el conjunto de validación. Por el contrario, el desempeño en entrenamiento empeoró, probablemente debido a una mayor dificultad de optimización sin un beneficio real en representación.

Tras experimentar con distintas combinaciones de épocas y tamaños de batch, se concluyó que la configuración más adecuada fue la que utilizó 500 épocas con un batch size de 128. Esta configuración logró un buen equilibrio entre rendimiento en entrenamiento y validación, mostrando señales de haber aprendido patrones útiles sin caer en un sobreajuste severo.

Aunque en configuraciones más moderadas (como 100 épocas y batch size de 32) el valor de pérdida (loss) en validación se mantuvo por debajo de 1.6, el desempeño en el conjunto de prueba fue menos consistente, con un loss cercano a 0.7 y una exactitud estimada de 0.75.

En contraste, con 500 épocas y batch de 128, si bien la pérdida en validación alcanzó valores de hasta 2.0, el modelo logró una pérdida en entrenamiento cercana a 0.25 y una exactitud muy cercana a 1.0. Esto indica que el modelo fue capaz de aprender profundamente del conjunto de entrenamiento y, a pesar de un mayor loss en validación, el sobreajuste no fue extremo, manteniendo un rendimiento competitivo en el conjunto de prueba.

Otro factor decisivo en el desempeño del modelo fue el desbalance de clases. La mayoría de las reseñas se concentran entre 3 y 5 estrellas, mientras que hay muy pocos ejemplos con 1 o 2 estrellas. Esto pudo haber llevado al modelo a aprender a predecir principalmente las clases mayoritarias, ignorando las minoritarias. Además, la longitud de las secuencias —hasta 348 tokens— representa otro desafío, ya que un MLP no está diseñado para manejar relaciones a largo plazo ni estructuras secuenciales complejas. En conjunto, estos factores refuerzan la conclusión de que, aunque el MLP fue capaz de memorizar el conjunto de entrenamiento, no logra generalizar de manera efectiva a nuevas reseñas.

Segunda Entrega

Experimentos

Se plantearon dos enfoques experimentales para abordar el problema de clasificación de reseñas, diferenciados por la forma en que se interpretan las etiquetas y se define la tarea de predicción.

Clasificación Binaria

En este primer enfoque, se reformuló el problema como una tarea de clasificación binaria, donde el objetivo es distinguir entre reseñas **negativas** y **positivas**. Para ello, se agruparon las clases originales de la siguiente manera:

- Clases 1, 2 y 3 estrellas → Clase 0: Negativa
- Clases 4 y 5 estrellas → Clase 1: Positiva

Clasificación Multiclase

En un segundo conjunto de experimentos, se mantuvieron las etiquetas originales de 1 a 5 estrellas, abordando el problema como una tarea de clasificación multiclase con cinco categorías:

- Clase 0: 1 estrella
- Clase 1: 2 estrellas
- Clase 2: 3 estrellas
- Clase 3: 4 estrellas
- Clase 4: 5 estrellas

Preprocesamiento

Antes de alimentar las reseñas al modelo, se adoptó un flujo de limpieza y vectorización rediseñado respecto a la Primera Entrega, combinando *spaCy* y utilidades de *Keras* para obtener un formato numérico coherente y libre de ruido.

- **Lematización y filtrado:** Con *spaCy* se realiza una lematización fina, eliminando stop-words y signos de puntuación. Así, cada reseña conserva solo los tokens con contenido semántico relevante.
- **Etiquetado flexible:** El pipeline admite clasificación binaria o multiclase (1-5 estrellas), según la configuración proporcionada.
- **Balanceo de clases:** Para mitigar el desbalance, se realiza:
    - Submuestreo
    - Uso de *class_weight* durante el entrenamiento.
- **División estratificada del conjunto:** Se aplica train_test_split con las siguientes proporciones:
  81 % entrenamiento · 9 % validación · 10 % prueba, garantizando que todas las clases están representadas en cada partición.

- **Longitud máxima de secuencia:** Se calcula el percentil 85 de las longitudes. Este valor se utiliza en la capa *TextVectorization* para truncar o rellenar secuencias, evitando entradas vacías y errores de *batching*.
- **Vectorización y *embeddings:*** Las reseñas se convierten en un *Dataset* de *Tensorflow*. La capa *TextVectorization* indexa el vocabulario y produce tensores de enteros, que luego pasan a una capa de *embeddings*. Esto asegura representaciones densas y continuas aptas para modelos secuenciales.
- **Persistencia del dataset:** El corpus procesado se guarda en formato *.parquet*, evitando descargas repetidas desde Kaggle y el costoso análisis lingüístico con *spaCy* en ejecuciones posteriores.

Esta nueva ruta de preprocesamiento asegura que los modelos secuenciales reciban entradas coherentes y sin secuencias vacías, reduciendo errores de batching y mejorando la estabilidad del entrenamiento.

Además, se buscó maximizar la proporción de datos destinados al entrenamiento (81 %), con el objetivo de proporcionar al modelo una mayor cantidad de ejemplos para el aprendizaje.

Por otro lado, se diseñó el preprocesamiento dentro de una función reutilizable con el fin de evitar la repetición de código y facilitar su integración en ciclos experimentales automáticos.

Desbalanceo

Uno de los principales desafíos encontrados durante el entrenamiento de modelos de clasificación fue el desbalance significativo en la distribución de clases, tanto en tareas binarias cómo multiclase.

En la versión binaria, las reseñas positivas representaban aproximadamente el 69.5 % del total, frente a un 30.5 % de reseñas negativas:

- Clase 0 – Negativa: 3136 ejemplos
- Clase 1 – Positiva: 6862 ejemplos

Este desbalance puede inducir al modelo a sesgarse hacia la clase mayoritaria, resultando en una alta exactitud superficial y un sobreajuste.

En la clasificación multiclase (de 1 a 5 estrellas), el desbalance se intensifica. Las clases extremas negativas están claramente subrepresentadas frente a las opiniones más positivas:

- Clase 0 – ★☆☆☆☆: 748 ejemplos
- Clase 1 – ★★☆☆☆: 927 ejemplos
- Clase 2 – ★★★☆☆: 1461 ejemplos
- Clase 3 – ★★★★☆: 3525 ejemplos
- Clase 4 – ★★★★★: 3337 ejemplos

Esto puede provocar que el modelo ignore por completo las clases menos frecuentes, dificultando su generalización.

Para mitigar el impacto de este desbalance, se diseñaron y compararon tres variantes de preprocesamiento:

1. **Sin balanceo:** Conjuntos originales, tal como aparecen en el dataset. Utilizado como referencia de base.
2. **Balanceado:** Se aplicó submuestreo de la clase mayoritaria para igualar la cantidad de ejemplos por clase. Esto generó datasets balanceados, por ejemplo:
    - En binaria: ~3000 ejemplos por clase en train, val y test.
    - En multiclase: ~700–900 ejemplos por clase.
3. **Ponderado:** Se mantuvieron los datos originales, pero se calcularon pesos inversamente proporcionales a la frecuencia de cada clase, aplicados en la función de pérdida.

RNN

Modelo

Las redes neuronales recurrentes (RNN) son una clase de modelos diseñados específicamente para procesar datos secuenciales. A diferencia de las redes MLP, las RNN incorporan conexiones recurrentes que permiten mantener un estado interno a lo largo del tiempo, lo cual les permite capturar dependencias contextuales entre palabras en una secuencia. Esta capacidad de modelar relaciones temporales las hace especialmente adecuadas para tareas de clasificación de texto, donde el significado de una palabra depende de su contexto previo.

La arquitectura utilizada fue optimizada automáticamente utilizando *Keras Tuner* con búsqueda bayesiana, explorando múltiples combinaciones de hiperparámetros para mejorar el rendimiento en clasificación binaria.

El modelo RNN implementado consistió en:

- **Capa de vectorización de texto**
  Transforma las reseñas en secuencias de enteros, donde cada entero representa un token del vocabulario. Esta transformación se realiza previamente usando *TextVectorization*.
- **Capa de Embedding**
  Proyecta cada índice de token a un vector denso. La dimensión de este espacio de embeddings fue ajustada automáticamente (valores posibles: 32, 64, 128, 256), junto con diferentes esquemas de inicialización (*glorot_uniform*, *he_normal*, etc.).
- **Capa(s) recurrente(s)**
  Se utilizaron entre 1 y 5 capas *SimpleRNN*, con número de unidades, *dropout* y *recurrent dropout* seleccionados por el tuner. Además, se aplicaron regularizadores L1/L2 opcionales, normalización de capas (LayerNormalization) y diferentes inicializadores para los pesos y estados recurrentes.
- **Capa densa intermedia (opcional)**
  En algunos modelos, se insertó una capa densa con activación (*ReLU*, *Tanh* o *GELU*), acompañada opcionalmente de *BatchNormalization* y *Dropout* adicional para mejorar la regularización.
- **Capa de salida**

- Para clasificación binaria, se usó una capa densa con activación *sigmoid*, que produce una probabilidad de clase positiva.
- Para clasificación multiclase, se utilizó una capa densa con activación *softmax* de 5 unidades, correspondiente a las clases de 1 a 5 estrellas.

Configuraciones:

- **Optimización con Keras Tuner**
  Se exploraron hasta 30 combinaciones de hiperparámetros por configuración experimental, buscando maximizar la métrica de validación (*val_accuracy*).
- **Función de pérdida:**
  - *Binary Crossentropy*, adecuada para clasificación binaria.
  - *Sparse Categorical Crossentropy* para clasificación multiclase (con etiquetas enteras 0–4).
- **Optimizador:**
  Adam, *SGD* y *RMSprop*, con ajustes individuales en tasa de aprendizaje, momento, etc.
- **Regularización:**
  Se aplicaron técnicas como *Dropout*, *SpatialDropout1D*, regularización *L2* y *L1L2*, dependiendo de los valores seleccionados por el tuner. Estas medidas buscaron reducir el sobreajuste y mejorar la generalización.
- **Batch size:**
  Fijo en 16, elegido por estabilidad durante la búsqueda de hiperparámetros.
- **Épocas:**
  El tuner entrenó por hasta 50 épocas con parada temprana, y luego se reentrenó el mejor modelo por 100 épocas adicionales con los mejores hiperparámetros encontrados.

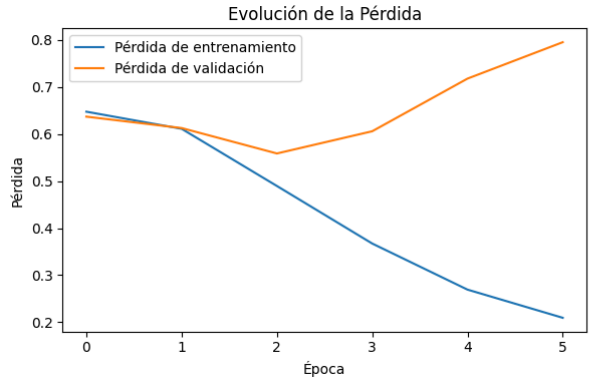
Resultados

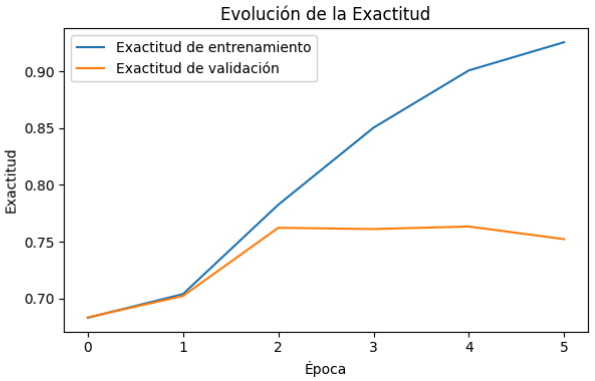
Binario

- **Sin balanceo**

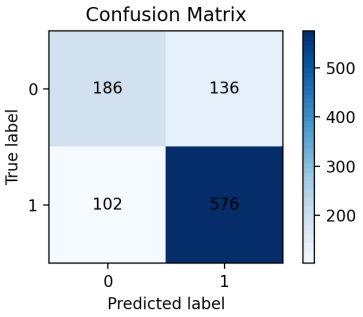
Hiperparámetro	**Valor**
embedding_dim	64
num_rnn_layers	2
rnn_units_0	32
rnn_units_1	256
optimizer	rmsprop
rmsprop_lr	2.15e-05

Test Loss: 0.52 | Test Accuracy: 0.76





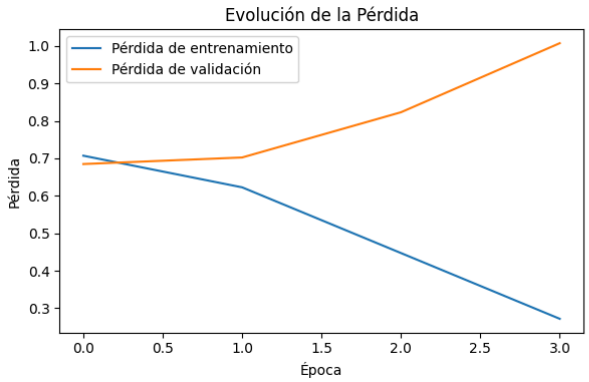
Class	Precision	Recall	F1-score	Support
0	0.646	0.578	0.61	322.0
1	0.809	0.85	0.829	678.0
accuracy	0.762	0.762	0.762	0.762
macro avg	0.727	0.714	0.719	1000.0
weighted avg	0.756	0.762	0.758	1000.0

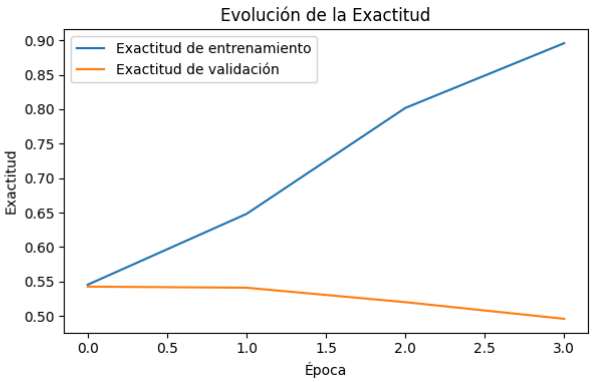


- **Balanceado**

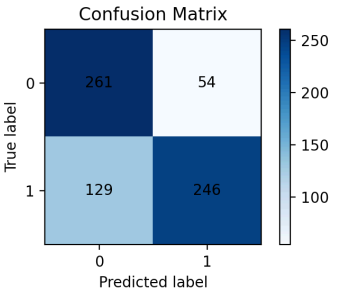
Hiperparámetro	**Valor**
embedding_dim	256
num_rnn_layers	1
rnn_units_0	64
optimizer	adam
adam_lr	5.16e-05

Test Loss: 0.57 | Test Accuracy: 0.73





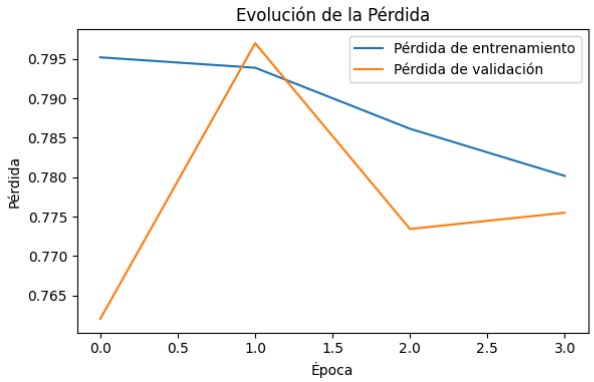
Class	Precision	Recall	F1-score	Support
0	0.669	0.829	0.74	315.0
1	0.82	0.656	0.729	375.0
accuracy	0.735	0.735	0.735	0.735
macro avg	0.745	0.742	0.735	690.0
weighted avg	0.751	0.735	0.734	690.0

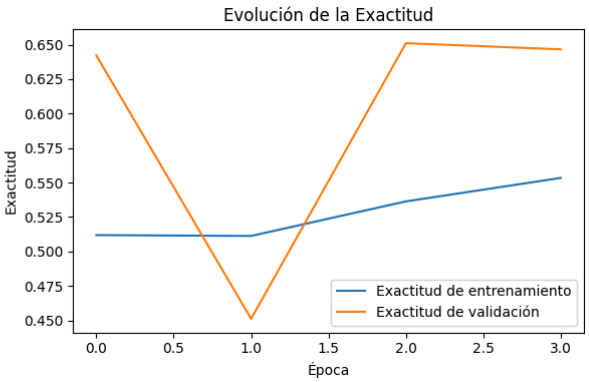


- **Ponderado**

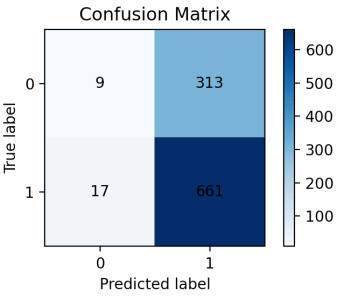
Hiperparámetro	**Valor**
embedding_dim	64
num_rnn_layers	4
rnn_units_0	192
rnn_units_1	64
rnn_units_2	192
rnn_units_4	32
dense_units	80
optimizer	sgd
sgd_lr	0.00035

Test Loss: 0.77 | Test Accuracy: 0.67





Class	Precision	Recall	F1-score	Support
0	0.346	0.028	0.052	322.0
1	0.679	0.975	0.8	678.0
accuracy	0.67	0.67	0.67	0.67
macro avg	0.512	0.501	0.426	1000.0
weighted avg	0.572	0.67	0.559	1000.0

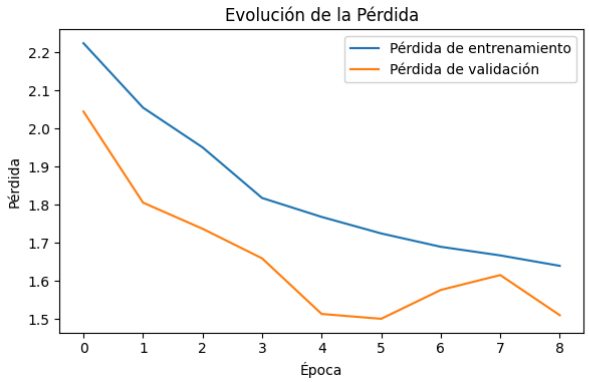


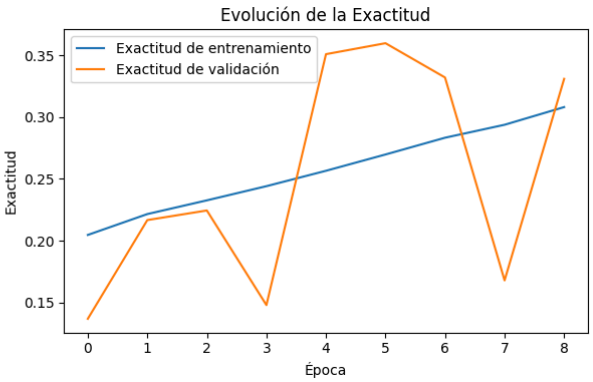
Multiclase

- **Sin balanceo**

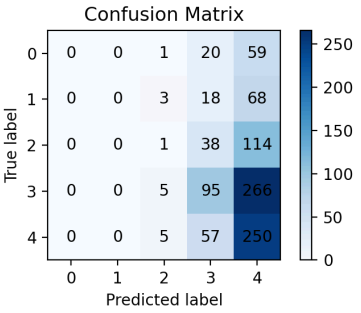
Hiperparámetro	**Valor**
embedding_dim	128
num_rnn_layers	2
rnn_units_0	256
rnn_units_1	64
dense_units	80
optimizer	adam
adam_lr	2.76e-05

- **Test Loss: 1.51 | Test Accuracy: 0.34**





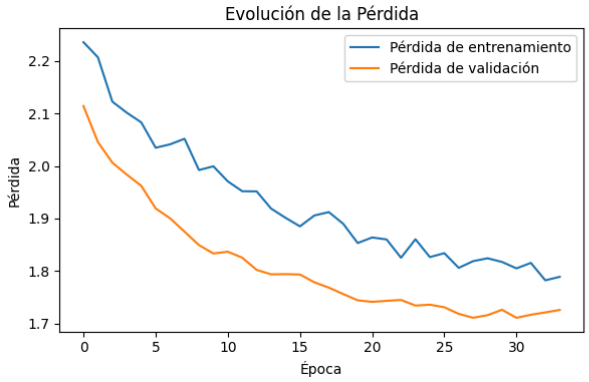
Class	Precision	Recall	F1-score	Support
0	0.0	0.0	0.0	80.0
1	0.0	0.0	0.0	89.0
2	0.067	0.007	0.012	153.0
3	0.417	0.26	0.32	366.0
4	0.33	0.801	0.468	312.0
accuracy	0.346	0.346	0.346	0.346
macro avg	0.163	0.213	0.16	1000.0
weighted avg	0.266	0.346	0.265	1000.0

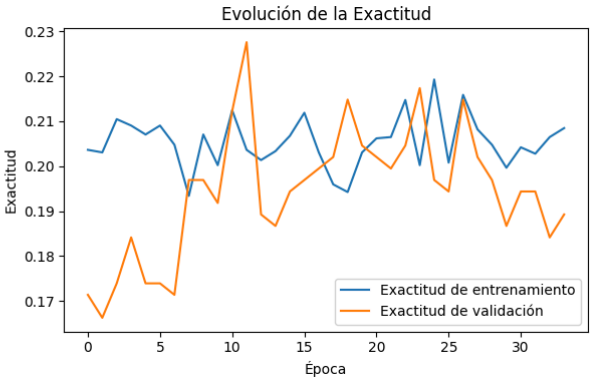


- **Balanceado**

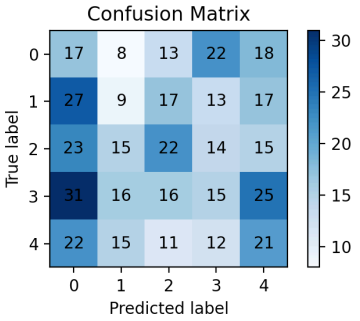
Hiperparámetro	**Valor**
embedding_dim	64
num_rnn_layers	2
rnn_units_0	160
rnn_units_1	128
optimizer	adam
adam_lr	1.36e-05

- **Test Loss: 1.70 | Test Accuracy: 0.19**





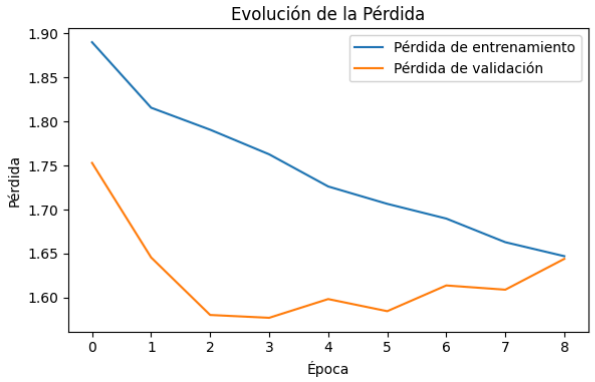
Class	Precision	Recall	F1-score	Support
0	0.142	0.218	0.172	78.0
1	0.143	0.108	0.123	83.0
2	0.278	0.247	0.262	89.0
3	0.197	0.146	0.168	103.0
4	0.219	0.259	0.237	81.0
accuracy	0.194	0.194	0.194	0.194
macro avg	0.196	0.196	0.192	434.0
weighted avg	0.198	0.194	0.192	434.0

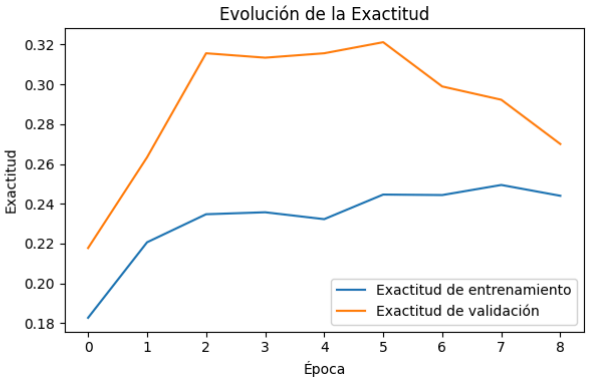


- **Ponderado**

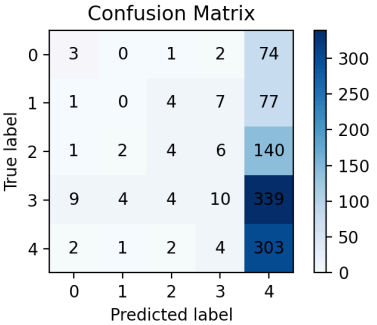
Hiperparámetro	**Valor**
embedding_dim	256
num_rnn_layers	3
rnn_units_0	128
rnn_units_1	32
rnn_units_2	96
optimizer	adam
adam_lr	1.09e-05

- **Test Loss: 1.58 | Test Accuracy: 0.32**





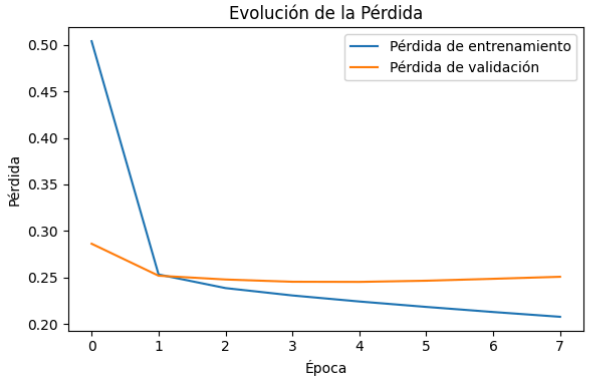
Class	Precision	Recall	F1-score	Support
0	0.188	0.038	0.062	80.0
1	0.0	0.0	0.0	89.0
2	0.267	0.026	0.048	153.0
3	0.345	0.027	0.051	366.0
4	0.325	0.971	0.487	312.0
accuracy	0.32	0.32	0.32	0.32
macro avg	0.225	0.212	0.129	1000.0
weighted avg	0.283	0.32	0.183	1000.0

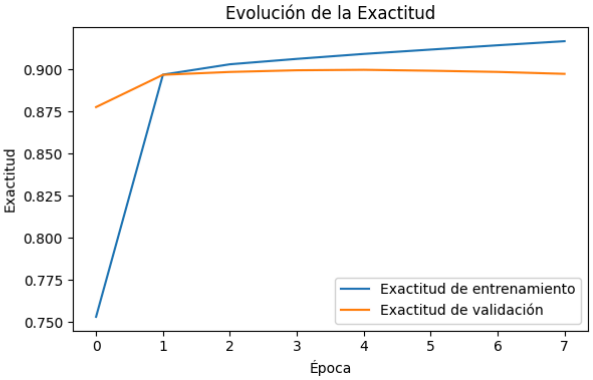


Mejores Modelos

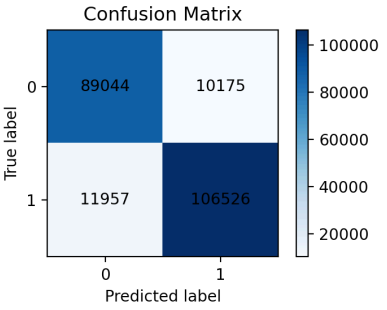
3 millones de datos.

Binario Balanceado

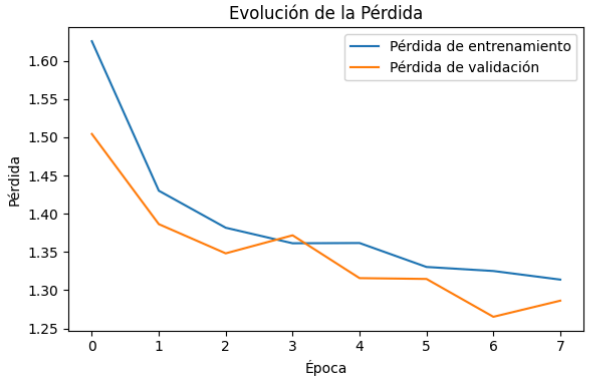


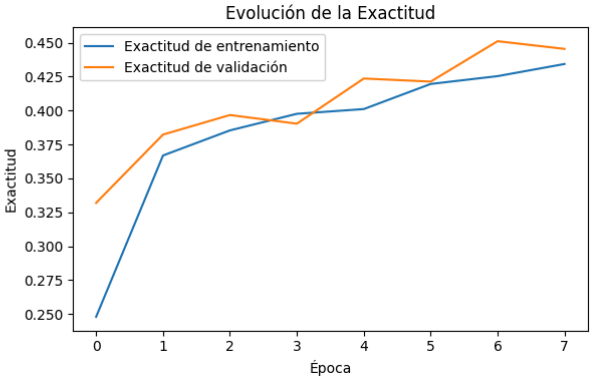


Class	Precision	Recall	F1-score	Support
0	0.882	0.897	0.889	99219.0
1	0.913	0.899	0.906	118483.0
accuracy	0.898	0.898	0.898	0.898
macro avg	0.897	0.898	0.898	217702.0
weighted avg	0.899	0.898	0.898	217702.0

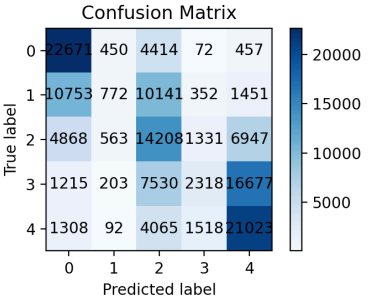


Multiclase Balanceado





Class	Precision	Recall	F1-score	Support
0.0	0.555	0.808	0.658	28064.0
1.0	0.371	0.033	0.06	23469.0
2.0	0.352	0.509	0.416	27917.0
3.0	0.415	0.083	0.138	27943.0
4.0	0.452	0.751	0.564	28006.0
accuracy	0.45	0.45	0.45	0.45
macro avg	0.429	0.437	0.367	135399.0
weighted avg	0.431	0.45	0.378	135399.0



Análisis

Binario

El modelo RNN fue evaluado bajo tres configuraciones distintas para abordar el desbalance de clases en un problema binario: sin balanceo, con balanceo estricto y mediante ponderación.

En primer lugar, en el enfoque sin balanceo, el modelo alcanzó una precisión del 76,2 % y una pérdida de 0.52, mostrando un desempeño competitiv.. Sin embargo, las gráficas de entrenamiento revelan un sobreajuste claro. Al observar la matriz de confusión y las métricas por clase, se identifica un sesgo hacia la clase 1, presentando un F1-score de 0.829, más alto que el 0.61 obtenido por la clase 0. Este patrón se debe a la falta de estrategias para mitigar el desbalance de clases, sumado al uso de RMSprop como optimizador, el cual suele tener un comportamiento menos robusto que Adam en escenarios complejos.

Por su parte, el modelo balanceado, el modelo alcanzó una precisión del 73 % y una pérdida de 0.57. No obstante, las gráficas de entrenamiento evidencian un sobreajuste aún más notorio que en el caso anterior. Además, el balanceo redujo la cantidad total de datos disponibles, lo que también puede limitar la capacidad de generalización. Al analizar la matriz de confusión y los F1-scores, se observa un rendimiento mucho más equilibrado entre clases: la clase 0 alcanza un F1-score de 0.74 y la clase 1 de 0.729. Esto indica que el modelo logró clasificar de forma más justa ambas clases, lo cual también se refleja en los promedios macro y ponderados. Llama la atención, además, que a pesar de que en las gráficas de entrenamiento la precisión de validación apenas ronda el 50 %, el desempeño sobre el conjunto de prueba se eleva hasta el 73 %, lo que podría deberse a una distribución más favorable de los datos de test.

Por otro lado, la configuración ponderada fue la que obtuvo el peor desempeño global entre las tres variantes evaluadas. A pesar de haber alcanzado una precisión aparente del 67 % y una pérdida de 0.77, las métricas revelan que el modelo incurrió en errores, con el F1-score más bajo de todas las configuraciones. La matriz de confusión evidencia que el modelo clasificó casi todas las instancias como pertenecientes a la clase 1. Esta desproporción sugiere que el modelo ignoró por completo a la clase minoritaria, a pesar de haber utilizado pesos compensatorios. Además, aunque no se observa un patrón clásico de sobreajuste en las curvas de entrenamiento, la validación presenta una inestabilidad notoria, con oscilaciones constantes que podrían reflejar dificultades de convergencia.

Finalmente, el modelo binario balanceado entrenado con 3 millones de ejemplos obtuvo el mejor desempeño entre todas las configuraciones evaluadas, alcanzando una precisión global del 89,8 % y un F1-score macro de 0.898. Este rendimiento sobresaliente se logró gracias a una combinación de volumen masivo de datos balanceados. Las curvas de entrenamiento muestran un patrón correcto, con una pérdida de validación que disminuye progresivamente y una exactitud que se estabiliza cerca del 90 %, sin evidencias claras de sobreajuste. Además, las métricas por clase reflejan una equidad notable en la clasificación: la clase 0 alcanzó un F1-score de 0.889 y la clase 1 de 0.906l. La matriz de confusión corrobora este equilibrio, mostrando una distribución casi simétrica de verdaderos positivos y bajos niveles de falsos positivos y negativos.

Multiclase

En el caso multiclase, el modelo RNN fue evaluado también bajo tres configuraciones: sin balanceo, con balanceo estricto y con ponderación de clases.

En primer lugar, en el enfoque sin balanceo obtuvo una precisión global del 34 % y una pérdida de 1.51, siendo el que mejor desempeño alcanzó. No obstante, un análisis de las métricas revela limitaciones. Por un lado, la gráfica de pérdida no evidencia un sobreajuste evidente, pero la curva de exactitud muestra un comportamiento errático, con subidas y caídas abruptas en la validación, lo que sugiere inestabilidad durante el entrenamiento. A esto se suma una matriz de confusión altamente sesgada: el modelo tiende a clasificar la mayoría de las muestras como pertenecientes a las clases 3 y 4. Este patrón de predicción deja en evidencia que, aunque el modelo logró cierta capacidad de ajuste, se vio fuertemente dominado por el desbalance de clases, resultando en un rendimiento desproporcionado.

Por su parte, el modelo balanceado fue el de peor desempeño entre los enfoques evaluados, alcanzando apenas un 19 % de exactitud y una pérdida de 1.70. Para esta configuración se aplicó submuestreo con el objetivo de equilibrar la cantidad de ejemplos por clase; sin embargo, esta estrategia redujo el volumen total de datos, lo que probablemente limitó severamente la capacidad del modelo para aprender patrones. Aunque la gráfica de pérdida no sugiere sobreajuste evidente, la de exactitud presenta un comportamiento extremadamente errático, con picos y valles abruptos tanto en las curvas de entrenamiento como de validación. Esta falta de consistencia se refleja también en los resultados. Los valores de precisión, recall y F1-score por clase son muy bajos y bastante similares entre sí, mientras que la matriz de confusión revela que el modelo no logra establecer ninguna preferencia clara por clases dominantes ni minoritarias, como si las predicciones fueran emitidas al azar.

Por otro lado, el modelo con ponderación de clases obtuvo un rendimiento intermedio, con una precisión del 32 % y una pérdida de 1.58, superando ligeramente al modelo balanceado por submuestreo pero aún por debajo del enfoque sin balanceo. Al observar las gráficas, la curva de pérdida no evidencia sobreajuste en las primeras épocas, pero hacia el final del entrenamiento se nota que el modelo comienza a sobreajustarse. En contraste, la gráfica de exactitud muestra el comportamiento más estable entre los tres modelos multiclase, con una evolución más uniforme durante el entrenamiento; sin embargo, hacia la última época también se evidencia una caída abrupta en la curva de validación, indicando una pérdida repentina en la capacidad de generalización. Por otro lado, el análisis de la matriz de confusión revela que el modelo, a pesar de emplear ponderación de clases, incurrió en un sesgo severo hacia la clase 4.

Finalmente, el enfoque multiclase balanceado con un conjunto de datos masivo, logró mejorar significativamente su rendimiento con una precisión general del 45 % y una pérdida de 1.28, evidenciando un aprendizaje más efectivo. Las gráficas muestran un descenso constante de la pérdida y un aumento progresivo de la exactitud sin indicios claros de sobreajuste. Esta estabilidad también se refleja en la gráfica de exactitud, con un comportamiento ascendente continuo. En cuanto a la matriz de confusión y la tabla de métricas, se observa una mejora sustancial en la capacidad de discriminación del modelo).

Sin embargo, la matriz de confusión revela un comportamiento curioso, el modelo clasifica con mayor acierto los extremos y la clase central, pero muestra mayores confusiones entre las clases intermedias. Esto sugiere que, si bien logró aprender patrones bien definidos en los polos y el centro de la escala, aún tiene dificultades para diferenciar matices entre categorías, tendiendo a asignar las observaciones confusas hacia los extremos o el centro

General

En clasificación binaria, el modelo sin balanceo mostró un rendimiento aceptable pero con fuerte sesgo hacia la clase mayoritaria. El balanceo estricto mejoró la equidad entre clases a costa de algo de precisión, mientras que la ponderación fue la estrategia menos efectiva. Al escalar a tres millones de ejemplos balanceados, el modelo alcanzó su mejor desempeño, con un F1-score macro de 89.8 % y una clasificación equitativa.

En la tarea multiclase, los resultados fueron más limitados. El modelo sin balanceo obtuvo la mayor precisión , pero con sesgo hacia la clase 4. El balanceo redujo drásticamente el rendimiento por falta de datos, y la ponderación no corrigió el sesgo. Solo al entrenar con tres millones de datos balanceados se logró una mejora significativa, aunque persistieron errores en clases intermedias, más difíciles de distinguir.

Un aspecto importante a destacar es que, en todos los modelos multiclase, las curvas de pérdida y exactitud no evidenciaron signos de sobreajuste, lo cual sugiere un entrenamiento relativamente estable. Sin embargo, esta ausencia de sobreajuste no se tradujo en buen rendimiento, ya que las precisiones fueron consistentemente bajas y los errores altos. Esto indica que el modelo RNN, aunque no se sobreajuste, tampoco logra aprender patrones suficientemente representativos para tareas complejas de clasificación multiclase.

En términos generales, los RNN mostraron sus mejores resultados en la tarea binaria, donde la simplicidad del problema facilita el aprendizaje, siempre que se cuente con un volumen significativo de datos. No obstante, para tareas más complejas como la clasificación multiclase, el modelo demostró claras limitaciones, y aun con grandes cantidades de datos balanceados, sus resultados fueron modestos. Una posible causa de estas dificultades es la longitud considerable de las secuencias utilizadas, entre 70 y 79 tokens, lo que representa un reto para una arquitectura RNN, ya que estas tienden a perder contexto. Esto sugiere que los RNN pueden quedar rezagados frente a arquitecturas más modernas que se explorarán en secciones posteriores.

LSTM

Modelo

Las redes LSTM (*Long Short-Term Memory*) son una extensión de las RNN diseñadas para superar sus limitaciones al capturar dependencias a largo plazo. Gracias a sus compuertas internas de entrada, olvido y salida, las LSTM pueden mantener información relevante durante secuencias largas, lo que las hace útiles en tareas de procesamiento del lenguaje natural donde el contexto completo de una oración influye en su interpretación.

La arquitectura utilizada fue optimizada automáticamente utilizando *Keras Tuner* con búsqueda bayesiana, explorando múltiples combinaciones de hiperparámetros para mejorar el rendimiento tanto en clasificación binaria como multiclase.

El modelo LSTM implementado consistió en:

- **Capa de vectorización de texto**
  Transforma las reseñas en secuencias de enteros, donde cada entero representa un token del vocabulario. Esta transformación se realiza previamente usando *TextVectorization*.
- **Capa de Embedding**
  Proyecta cada índice de token a un vector denso. La dimensión de este espacio de embeddings fue ajustada automáticamente (valores posibles: 32, 64, 128, 256), junto con diferentes esquemas de inicialización (*glorot_uniform*, *he_normal*, etc.).
- **Capa(s) recurrente(s)**
  Se utilizaron entre 1 y 5 capas *LSTM*, con número de unidades y *recurrent dropout* seleccionados por el tuner. Además, se aplicaron regularizadores *L1/L2* opcionales, normalización de capas y diferentes inicializadores para los pesos y estados recurrentes.
- **Capa densa intermedia (opcional)**
  En algunos modelos, se insertó una capa densa con activación (*ReLU*, *Tanh* o *GELU*), acompañada opcionalmente de *BatchNormalization* y *Dropout* adicional para mejorar la regularización.
- **Capa de salida**
  - Para clasificación binaria, se usó una capa densa con activación *sigmoid*, que produce una probabilidad de clase positiva.
  - Para clasificación multiclase, se utilizó una capa densa con activación *softmax* de 5 unidades, correspondiente a las clases de 1 a 5 estrellas.

Configuraciones:

- **Optimización con Keras Tuner**
  Se exploraron hasta 30 combinaciones de hiperparámetros por configuración experimental, buscando maximizar la métrica de validación (*val_accuracy*).
- **Función de pérdida:**
  - *Binary Crossentropy*, adecuada para clasificación binaria.
  - *Sparse Categorical Crossentropy* para clasificación multiclase (con etiquetas enteras 0–4).
- **Optimizador:**
  Adam, *SGD* y *RMSprop*, con ajustes individuales en tasa de aprendizaje, momento, etc.
- **Regularización:**
  Se aplicaron técnicas como *Dropout*, *SpatialDropout1D*, regularización *L2* y *L1L2*, dependiendo de los valores seleccionados por el tuner.
- **Batch size:**
  Fijo en 16.
- **Épocas:**
  El tuner entrenó por hasta 50 épocas con parada temprana, y luego se reentrenó el

mejor modelo por otras 50 épocas adicionales con los mejores hiperparámetros encontrados para obtener el histórico.

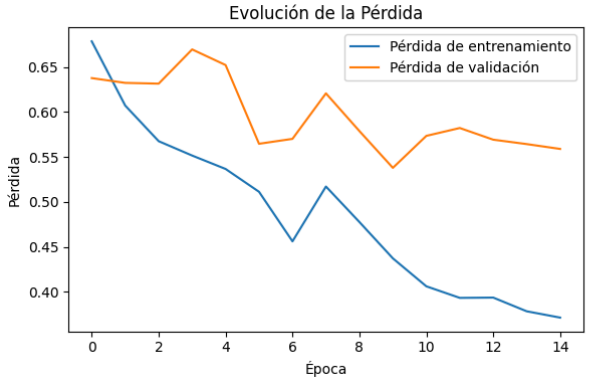
Resultados

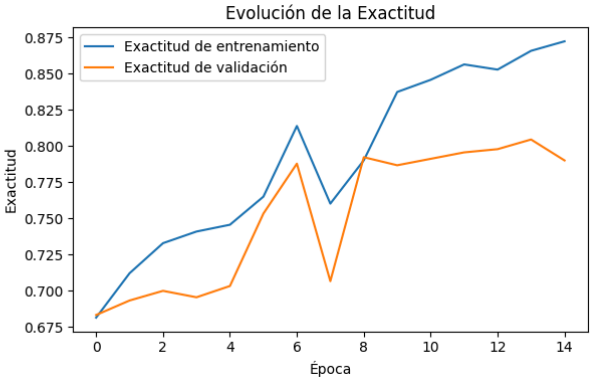
Binario

- **Sin balanceo**

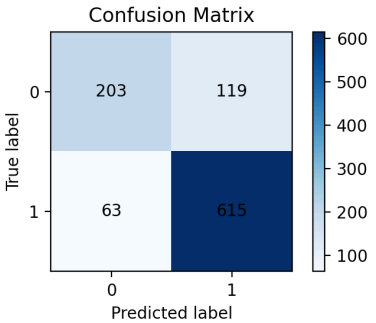
Hiperparámetro	**Valor**
embedding_dim	128
num_lstm_layers	1
lstm_units_0	32
optimizer	adam
adam_lr	0.001

Test Loss: 0.48 | Test Accuracy: 0.81





Class	Precision	Recall	F1-score	Support
0	0.763	0.63	0.69	322.0
1	0.838	0.907	0.871	678.0
accuracy	0.818	0.818	0.818	0.818
macro avg	0.801	0.769	0.781	1000.0
weighted avg	0.814	0.818	0.813	1000.0

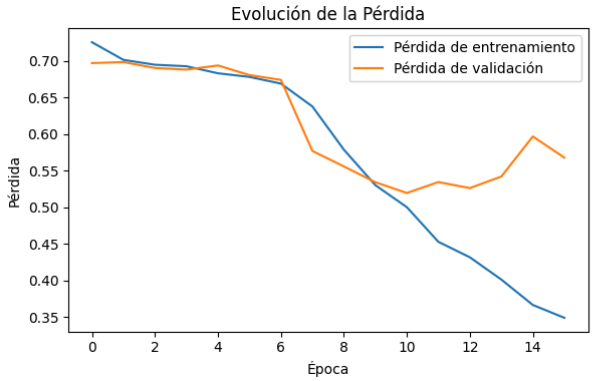


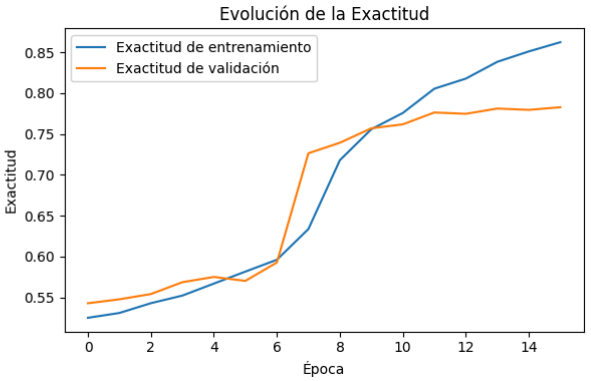
- **Balanceado**

Hiperparámetro	**Valor**
embedding_dim	64
num_lstm_layers	4

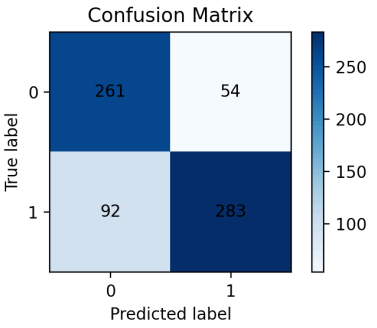
Hiperparámetro	Valor
lstm_units_0	64
lstm_units_1	160
lstm_units_2	160
lstm_units_3	96
optimizer	adam
adam_lr	0.00015

Test Loss: 0.54 | Test Accuracy: 0.78





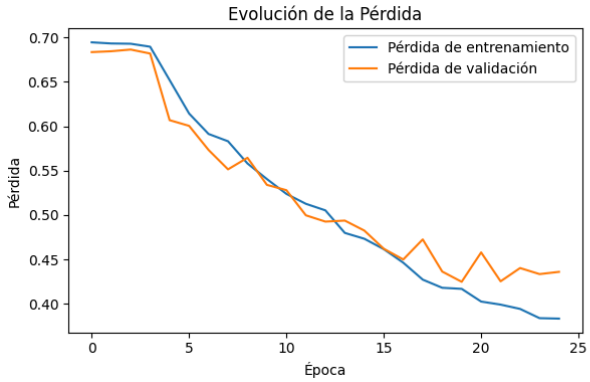
Class	Precision	Recall	F1-score	Support
0	0.739	0.829	0.781	315.0
1	0.84	0.755	0.795	375.0
accuracy	0.788	0.788	0.788	0.788
macro avg	0.79	0.792	0.788	690.0
weighted avg	0.794	0.788	0.789	690.0

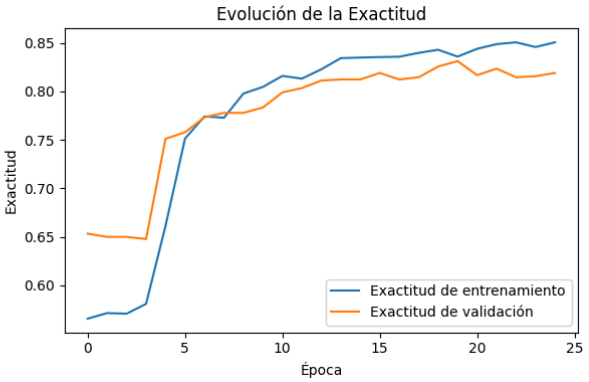


- **Ponderado**

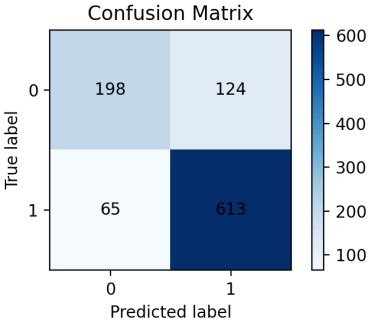
Hiperparámetro	Valor
embedding_dim	128
num_lstm_layers	1
lstm_units_0	32
optimizer	adam
adam_lr	0.00013

Test Loss: 0.45 | Test Accuracy: 0.81





Class	Precision	Recall	F1-score	Support
0	0.753	0.615	0.677	322.0
1	0.832	0.904	0.866	678.0
accuracy	0.811	0.811	0.811	0.811
macro avg	0.792	0.76	0.772	1000.0
weighted avg	0.806	0.811	0.805	1000.0

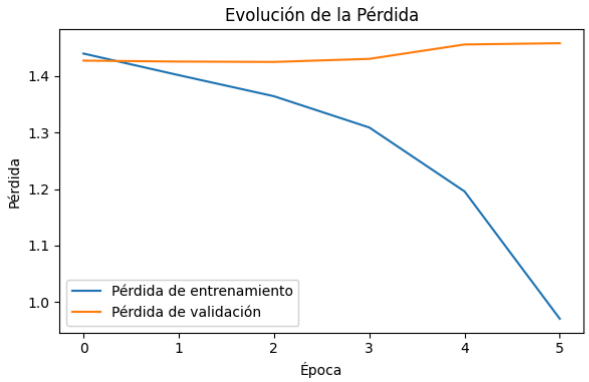


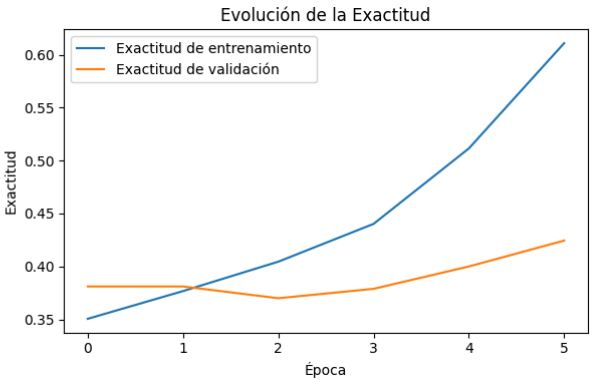
Multiclase

- **Sin balanceo**

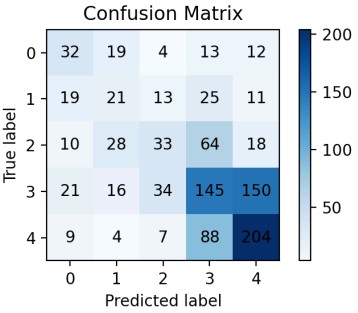
Hiperparámetro	**Valor**
embedding_dim	256
num_lstm_layers	1
lstm_units_0	64
optimizer	adam
rms_lr	5.16e-05

Test Loss:1.44 | Test Accuracy: 0.43





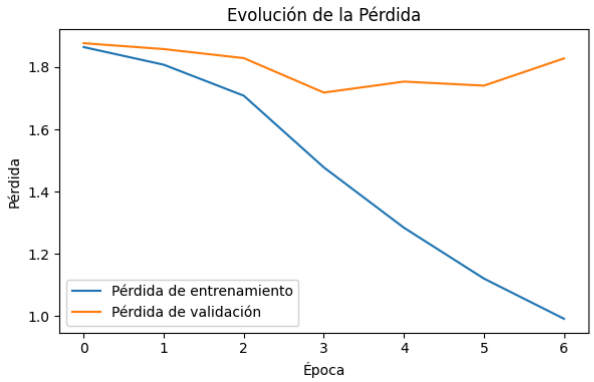
Class	Precision	Recall	F1-score	Support
0	0.352	0.4	0.374	80.0
1	0.239	0.236	0.237	89.0
2	0.363	0.216	0.27	153.0
3	0.433	0.396	0.414	366.0
4	0.516	0.654	0.577	312.0
accuracy	0.435	0.435	0.435	0.435
macro avg	0.38	0.38	0.375	1000.0
weighted avg	0.424	0.435	0.424	1000.0

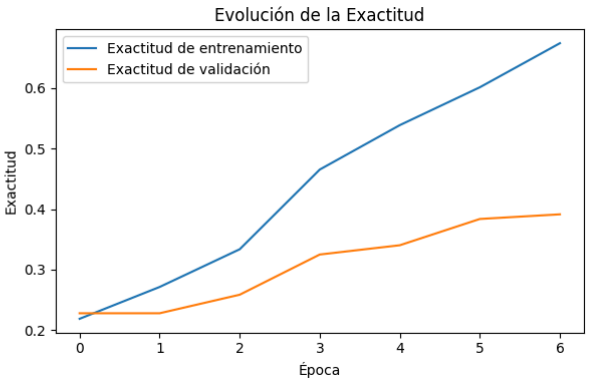


- **Balanceado**

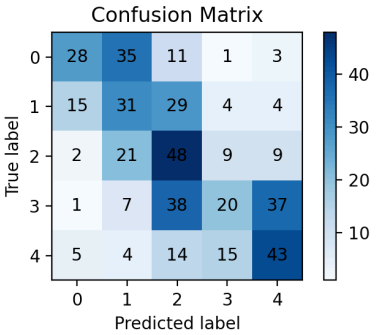
Hiperparámetro	**Valor**
embedding_dim	256
num_lstm_layers	5
lstm_units_0	256
lstm_units_1	96
lstm_units_2	192
lstm_units_3	160
lstm_units_4	160
optimizer	rmsprop
rms_lr	1.68e-05

Test Loss: 1.83 | Test Accuracy: 0.39





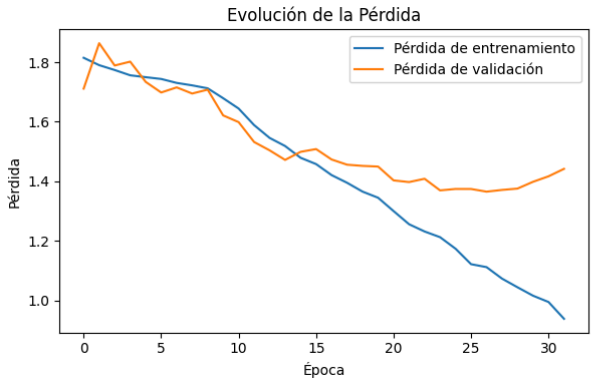
Class	Precision	Recall	F1-score	Support
0	0.549	0.359	0.434	78.0
1	0.316	0.373	0.343	83.0
2	0.343	0.539	0.419	89.0
3	0.408	0.194	0.263	103.0
4	0.448	0.531	0.486	81.0
accuracy	0.392	0.392	0.392	0.392
macro avg	0.413	0.399	0.389	434.0
weighted avg	0.41	0.392	0.383	434.0

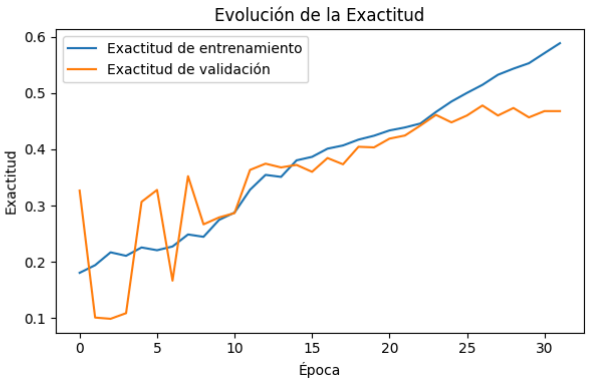


- **Ponderado**

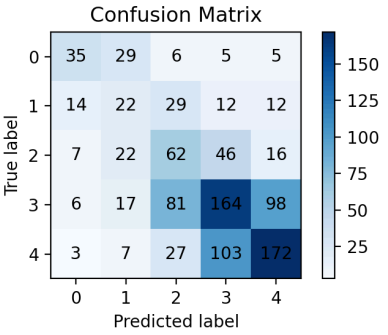
Hiperparámetro	**Valor**
embedding_dim	32
num_lstm_layers	5
lstm_units_0	192
lstm_units_1	160
lstm_units_2	96
lstm_units_3	160
lstm_units_4	96
optimizer	adam
adam_lr	1.53e-05

Test Loss: 1.49 | Test Accuracy: 0.45





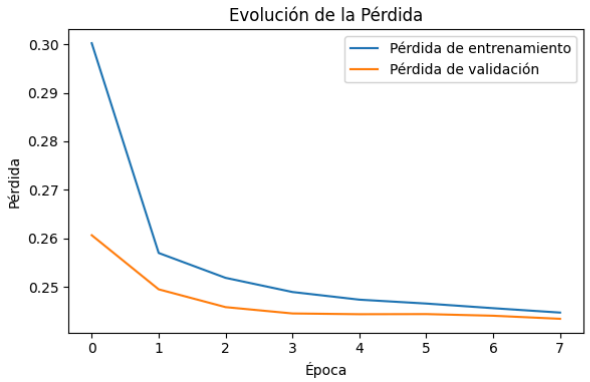
Class	Precision	Recall	F1-score	Support
0	0.538	0.438	0.483	80.0
1	0.227	0.247	0.237	89.0
2	0.302	0.405	0.346	153.0
3	0.497	0.448	0.471	366.0
4	0.568	0.551	0.559	312.0
accuracy	0.455	0.455	0.455	0.455
macro avg	0.426	0.418	0.419	1000.0
weighted avg	0.469	0.455	0.46	1000.0

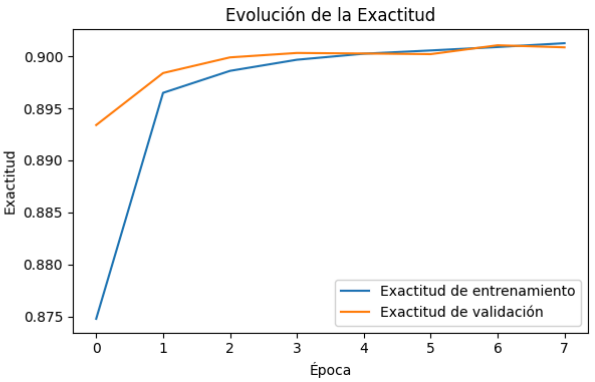


Mejores Modelos

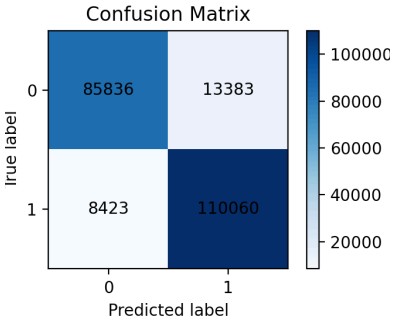
3 millones de datos.

Binario Balanceado

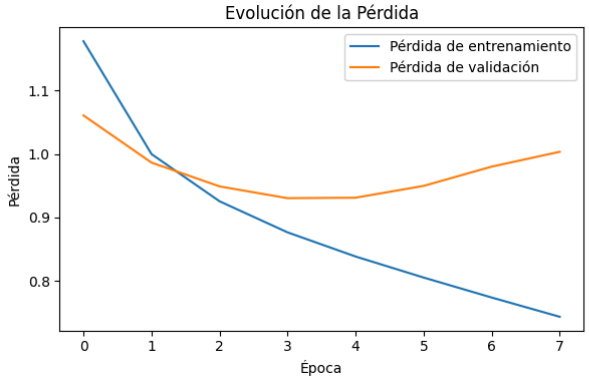


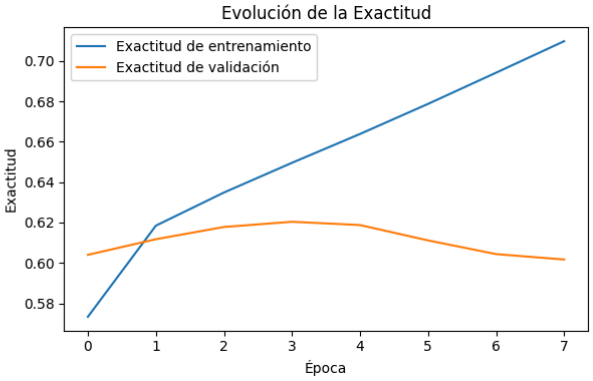


Class	Precision	Recall	F1-score	Support
0	0.911	0.865	0.887	99219.0
1	0.892	0.929	0.91	118483.0
accuracy	0.9	0.9	0.9	0.9
macro avg	0.901	0.897	0.899	217702.0
weighted avg	0.9	0.9	0.9	217702.0

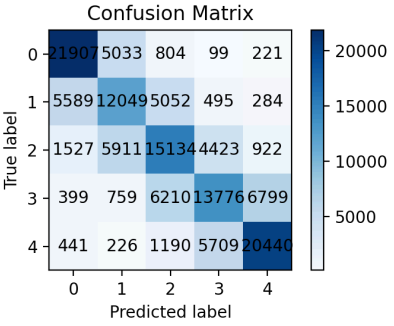


Multiclase Balanceado





Class	Precision	Recall	F1-score	Support
0.0	0.734	0.781	0.756	28064.0
1.0	0.503	0.513	0.508	23469.0
2.0	0.533	0.542	0.538	27917.0
3.0	0.562	0.493	0.525	27943.0
4.0	0.713	0.73	0.721	28006.0
accuracy	0.615	0.615	0.615	0.615
macro avg	0.609	0.612	0.61	135399.0
weighted avg	0.613	0.615	0.613	135399.0



Análisis

Binario

El modelo LSTM fue evaluado bajo tres configuraciones distintas para abordar el desbalance de clases en un problema binario: sin balanceo, con balanceo estricto y mediante ponderación.

En primer lugar, el enfoque sin balanceo alcanzó la mayor precisión en el conjunto de prueba, 81.8 %, y una pérdida de 0.4829. No obstante, este rendimiento aparentemente favorable estuvo acompañado por una divergencia entre las curvas de entrenamiento y validación a lo largo de las épocas, lo que sugiere un proceso de sobreajuste. Esta situación se debe al predominio de la clase mayoritaria, que induce al modelo a aprender patrones específicos de dicha clase, limitando su capacidad de generalización. Además, las gráficas de entrenamiento muestran una evolución inestable, con picos y caídas abruptas, lo que refuerza la idea de una falta de consistencia en el aprendizaje del modelo.

Por su parte, el modelo balanceado, experimentó una reducción notable en el rendimiento: la precisión descendió al 78 % y la pérdida aumentó a 0.54. Esta disminución puede atribuirse a la reducción en el tamaño del conjunto de entrenamiento. Aun así, esta configuración favorece una clasificación más equitativa entre clases, como lo reflejan las métricas de precisión, recall y F1-score, más niveladas que en los otros escenarios. Desde el punto de vista del entrenamiento, las curvas inician con un comportamiento coherente, pero luego se observa una separación entre entrenamiento y validación, junto con un aumento en la pérdida de validación, lo que indica un inicio de sobreajuste derivado de la escasez de datos.

Por otro lado, el modelo con ponderación de clases no logró mejorar significativamente el desbalance respecto al modelo sin balanceo; de hecho, lo empeoró ligeramente. Si bien alcanzó una exactitud del 81 % y una pérdida de 0.45, y presentó curvas de entrenamiento limpias y estables sin signos evidentes de sobreajuste, estos indicadores no se tradujeron en una mejora efectiva en la equidad de clasificación entre clases.

Finalmente, al escalar el experimento con un conjunto de datos significativamente mayor, el modelo balanceado evidenció una mejora tanto en precisión como en estabilidad. Se alcanzó una exactitud del 90 % y un F1-score de 0.899, reflejando una capacidad de generalización más robusta y una clasificación equitativa entre clases. Las curvas de pérdida y exactitud mostraron una convergencia rápida y consistente, sin señales de sobreajuste ni oscilaciones abruptas. Además, las métricas por clase se mantuvieron balanceadas, con un F1-score de 0.887 para la clase 0 y de 0.910 para la clase 1, lo que indica que el modelo logró mantener la equidad. Estos resultados sugieren que, al disponer de un volumen de datos suficientemente amplio, la estrategia de balanceo estricto deja de ser una limitación y puede aprovechar todo su potencial para entrenar modelos robustos y justos, con un buen rendimiento general.

Multiclase

En el caso multiclase, el modelo LSTM fue evaluado también bajo tres configuraciones: sin balanceo, con balanceo estricto y con ponderación de clases.

En primer lugar, el enfoque sin balanceo obtuvo la mayor exactitud en el conjunto de prueba (43 %) y la menor pérdida (1.44) en comparación con las demás configuraciones. No obstante, estas métricas deben interpretarse, ya que, al observar las gráficas de evolución de la pérdida, se evidencia un claro indicio de sobreajuste. Además, al analizar la matriz de confusión, se observa un marcado sesgo hacia las clases más representadas, particularmente las de 4 y 5 estrellas, mientras que las demás clases presentan una distribución de errores mucho más dispersa. En consecuencia, el modelo, aunque entrenado con todos los datos, tiende a reproducir el desequilibrio presente en la distribución original, clasificando de forma desproporcionada hacia las clases dominantes y descuidando las minoritarias.

Por otro lado, el modelo balanceado, entrenado con un número igual de muestras por clase, presentó un deterioro en su rendimiento, con una precisión de 39 % y una pérdida de 1.83. Esta caída se relaciona directamente con la reducción del volumen total de datos. Las gráficas de entrenamiento muestran, desde las primeras épocas, signos claros de sobreajuste. No obstante, al examinar la matriz de confusión, se perciben indicios de una intención de aprendizaje estructurado, reflejada en una diagonal parcialmente visible. A pesar de esta tendencia positiva, el modelo aún comete errores importantes, por ejemplo, confunde con frecuencia reseñas de 5 estrellas con las de 4, y las de 3 con clases vecinas como 2 o 4, lo que muestra que las predicciones tienden a desplazarse hacia clases adyacentes.

Por otro lado, el modelo con ponderación alcanzó un rendimiento intermedio, con una exactitud del 45 % y una pérdida de 1.49. Al observar las curvas de entrenamiento, se destaca como el modelo que tarda más en sobreajustarse, manteniendo una relación relativamente estable entre pérdida de entrenamiento y validación durante las primeras 15 a

20 épocas. Aunque eventualmente también incurre en sobreajuste, la transición es más gradual. No obstante, la curva de exactitud muestra mayor inestabilidad respecto a las otras configuraciones, lo cual podría atribuirse tanto a la sensibilidad del modelo ante las ponderaciones iniciales como al hecho de haber necesitado más épocas para detenerse, generando una gráfica más extensa. En cuanto a la matriz de confusión, se observa un patrón similar al del modelo sin balanceo, con una clara concentración de aciertos en las clases mayoritarias.

Finalmente, al escalar el experimento con un conjunto de datos significativamente mayor, se observó una mejora en el rendimiento general, alcanzando una exactitud del 61,5 % y valores de F1-score promedio de 0.610 y 0.615. Las curvas de entrenamiento reflejan un patrón de aprendizaje más sostenido, aunque nuevamente aparece el sobreajuste a partir de la cuarta época. No obstante, este sobreajuste es más controlado que en experimentos previos, lo que indica una mejor capacidad de generalización. La matriz de confusión evidencia una diagonal más definida, señal de que el modelo acierta con mayor frecuencia en la clasificación correcta, especialmente en las clases de los extremos. Esto puede explicarse porque dichas clases reflejan opiniones más polarizadas y características lingüísticas más marcadas, lo que facilita su diferenciación frente a las clases intermedias, cuyas expresiones suelen ser más ambiguas y difíciles de distinguir entre sí.

General

En términos generales, los resultados obtenidos muestran que el desempeño en clasificación binaria fue consistentemente superior al multiclase, lo cual es esperable dado que se trata de un problema más sencillo y menos afectado por el desbalance de clases, ya que los datos se agrupan únicamente en dos categorías. En contraste, el enfoque multiclase arrojó métricas considerablemente más bajas, incluso al aumentar significativamente el volumen de datos.

A nivel de comportamiento, todos los modelos siguieron una dinámica similar: los modelos balanceados lograron una distribución más equitativa de los aciertos entre las clases, pero con un costo en la exactitud global, debido a la drástica reducción del número de ejemplos disponibles. Sin embargo, al escalar el conjunto de datos, el modelo balanceado demostró un rendimiento sobresaliente en la tarea binaria y aceptable en multiclase, lo que indica que el balance estricto deja de ser una limitación cuando se dispone de datos masivos. Por otro lado, los modelos sin balanceo y ponderados, aunque alcanzaron mejores métricas globales, continuaron reflejando un claro sesgo hacia las clases mayoritarias.

En particular, el modelo ponderado no logró compensar del todo este efecto, a pesar de que su diseño buscaba precisamente corregirlo mediante la asignación de pesos. Esto sugiere que la estrategia de ponderación implementada podría no haber sido la más adecuada o que, durante el proceso de entrenamiento, el modelo terminó ignorando parcialmente esos pesos y adaptándose nuevamente a la distribución desbalanceada de la tarea.

Transformer

Los modelos Transformer han demostrado ser efectivos en tareas de procesamiento de lenguaje natural al capturar relaciones complejas y dependencias a largo plazo a través de mecanismos de atención. En este trabajo, se utilizó transferencia de conocimiento a partir

del modelo preentrenado RoBERTa-base, implementado con la librería Keras y ajustado para las tareas de clasificación propuestas.

La arquitectura utilizada se basó en el modelo RoBERTa-base preentrenado, aplicado mediante técnicas de fine-tuning para ajustar sus parámetros a las tareas de clasificación tanto binaria como multiclase. A diferencia de los modelos LSTM o RNN, el pipeline para RoBERTa no requiere capas explícitas de vectorización o embedding, ya que el propio modelo gestiona la tokenización y la representación contextual de los textos.

El modelo Transformer implementado consistió en:

- **Tokenización y embeddings contextuales**
  Las reseñas fueron convertidas automáticamente en secuencias de tokens y representadas como vectores contextuales mediante el tokenizador propio de RoBERTa.
- **Modelo base RoBERTa**
  Se utilizó el preset *roberta_base_en* de keras-hub.
- **Capa de salida**
  Se añadió una capa densa de salida sobre el embedding del modelo, con activación *sigmoid* para clasificación binaria y *softmax* para multiclase, correspondiendo a las estrellas 1–5.

Configuraciones:

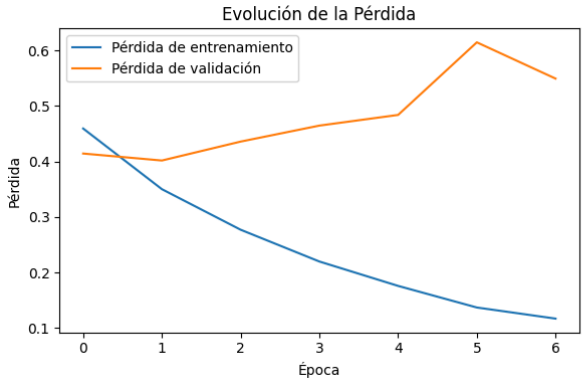
- **Optimización sin búsqueda exhaustiva de hiperparámetros:**
  Debido a las limitaciones de recursos computacionales en Google Colab y el alto costo de entrenamiento de los modelos Transformer, no se empleó Keras Tuner ni búsqueda bayesiana.
- **Función de pérdida:**
  - *Binary Crossentropy*, adecuada para clasificación binaria.
  - *Sparse Categorical Crossentropy* para clasificación multiclase (con etiquetas enteras 0–4).
- **Optimizadores:**
  Se utilizó el optimizador Adam con parámetros por defecto recomendados para fine-tuning de Transformers.
- **Batch size:**
  Fijo en 16, limitado por la memoria disponible en Colab.
- **Regularización:**
  Se aplicó el *early stopping* para evitar el sobreajuste.
- **Épocas:**
  Cada modelo se entrenó durante un máximo de 10 épocas, con un mecanismo de *early stopping* con paciencia de 5 épocas, deteniendo el entrenamiento si no se observaba mejora en la métrica de validación.

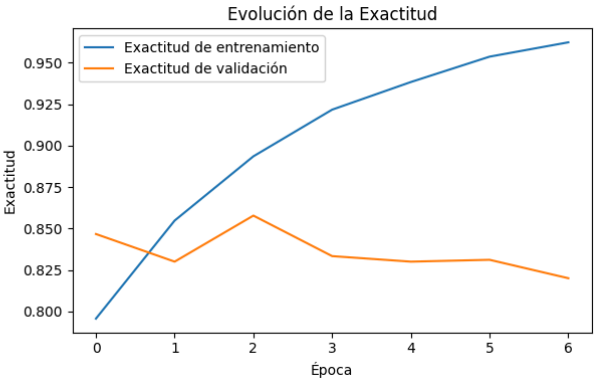
Modelo

Resultados

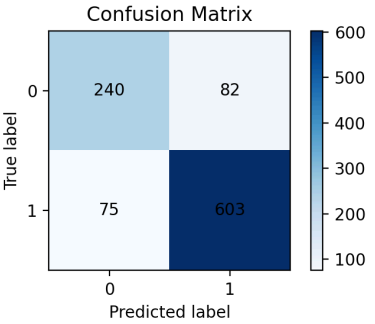
Binario

- **Sin balanceo**
  Test Loss:0.39 | Test Accuracy: 0.84

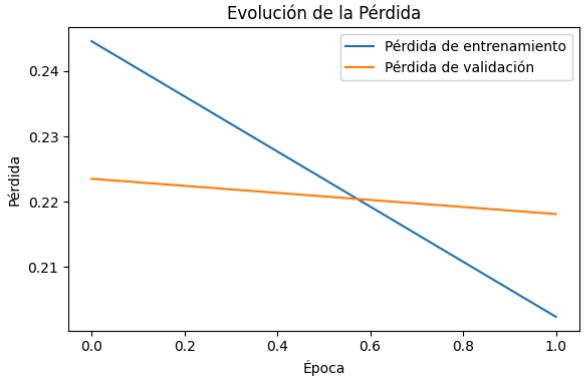


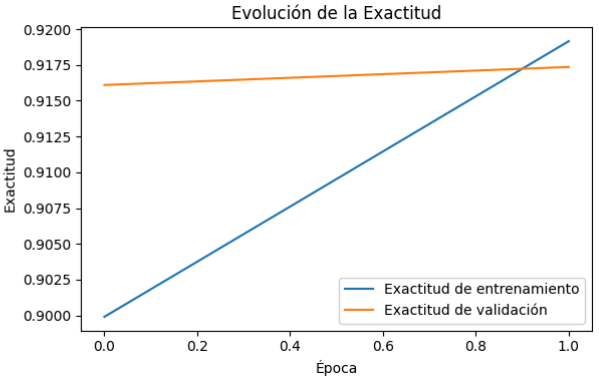


Class	Precision	Recall	F1-score	Support
0	0.762	0.745	0.754	322.0
1	0.88	0.889	0.885	678.0
accuracy	0.843	0.843	0.843	0.843
macro avg	0.821	0.817	0.819	1000.0
weighted avg	0.842	0.843	0.843	1000.0

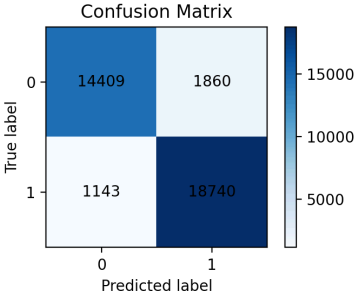


- **Balanceado (500 mil)**
  Test Loss: 0.22 | Test Accuracy: 0.91

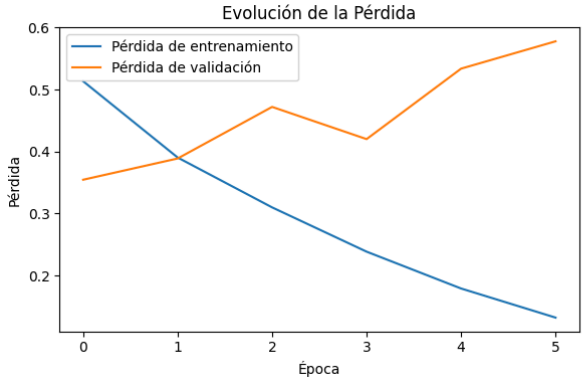


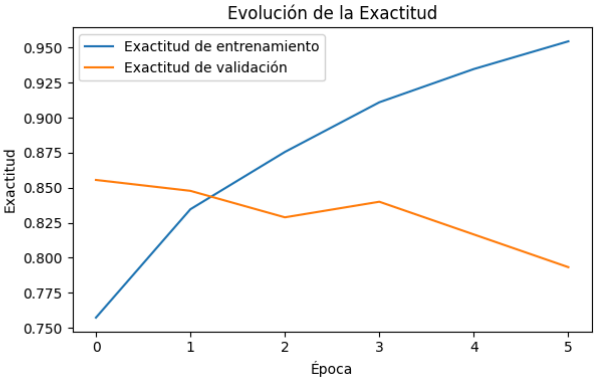


Class	Precision	Recall	F1-score	Support
0	0.927	0.886	0.906	16269.0
1	0.91	0.943	0.926	19883.0
accuracy	0.917	0.917	0.917	0.917
macro avg	0.918	0.914	0.916	36152.0
weighted avg	0.917	0.917	0.917	36152.0

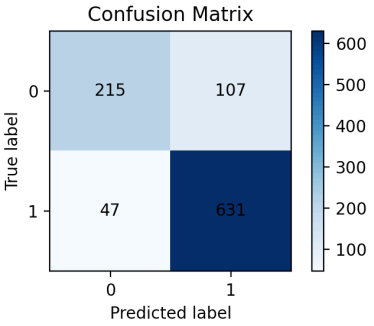


- **Ponderado**
  Test Loss:0.36 | Test Accuracy: 0.84



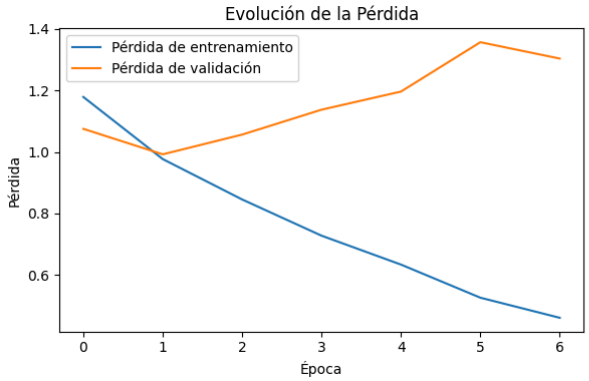


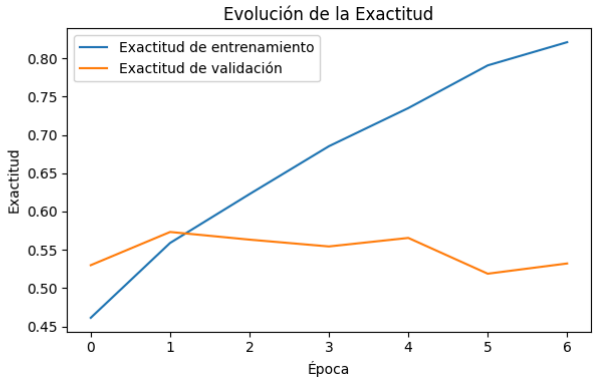
Class	Precision	Recall	F1-score	Support
0	0.821	0.668	0.736	322.0
1	0.855	0.931	0.891	678.0
accuracy	0.846	0.846	0.846	0.846
macro avg	0.838	0.799	0.814	1000.0
weighted avg	0.844	0.846	0.841	1000.0



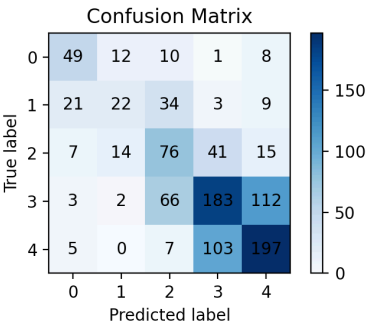
Multiclase

- **Sin balanceo**
  Test Loss: 1.02 | Test Accuracy: 0.52



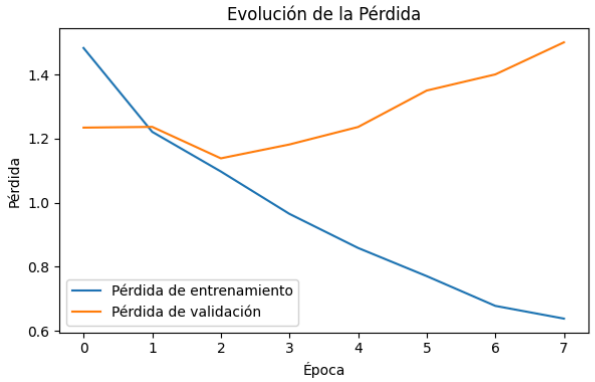


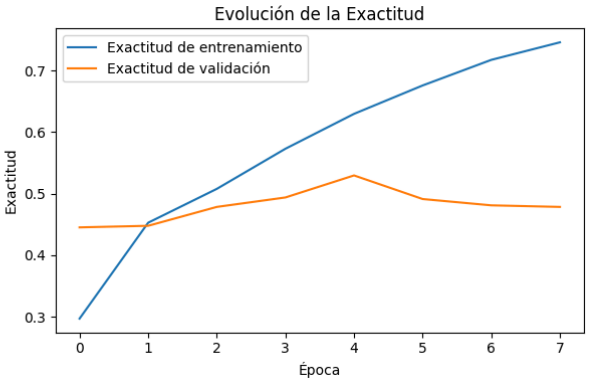
Class	Precision	Recall	F1-score	Support
0	0.576	0.612	0.594	80.0
1	0.44	0.247	0.317	89.0
2	0.394	0.497	0.439	153.0
3	0.553	0.5	0.525	366.0
4	0.578	0.631	0.603	312.0
accuracy	0.527	0.527	0.527	0.527
macro avg	0.508	0.498	0.496	1000.0
weighted avg	0.528	0.527	0.523	1000.0



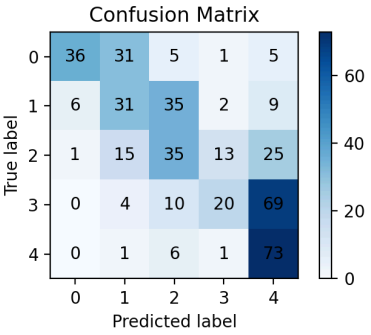
- **Balanceado**

Test Loss: 1.17 | Test Accuracy: 0.44



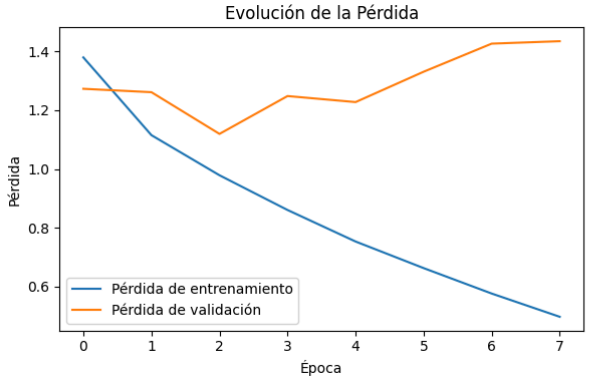


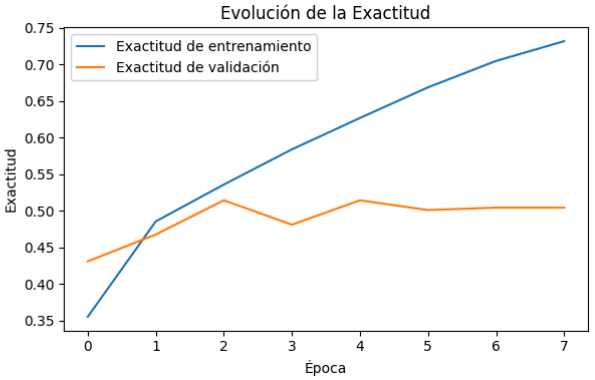
Class	Precision	Recall	F1-score	Support
0	0.837	0.462	0.595	78.0
1	0.378	0.373	0.376	83.0
2	0.385	0.393	0.389	89.0
3	0.541	0.194	0.286	103.0
4	0.403	0.901	0.557	81.0
accuracy	0.449	0.449	0.449	0.449
macro avg	0.509	0.465	0.441	434.0
weighted avg	0.505	0.449	0.43	434.0



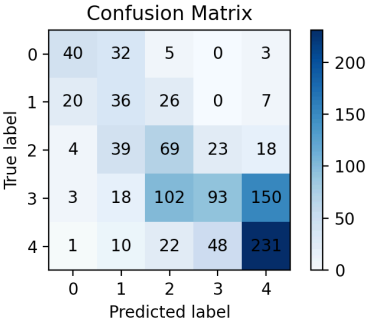
- **Ponderado**

  Test Loss: 1.14 | Test Accuracy: 0.46





Class	Precision	Recall	F1-score	Support
0	0.588	0.5	0.541	80.0
1	0.267	0.404	0.321	89.0
2	0.308	0.451	0.366	153.0
3	0.567	0.254	0.351	366.0
4	0.565	0.74	0.641	312.0
accuracy	0.469	0.469	0.469	0.469
macro avg	0.459	0.47	0.444	1000.0
weighted avg	0.502	0.469	0.456	1000.0



Análisis

Binario

El modelo Transformer fue evaluado bajo tres configuraciones distintas para abordar el desbalance de clases en un problema binario: sin balanceo, con balanceo estricto y mediante ponderación.

En la configuración sin balanceo, el modelo alcanzó una precisión del 84,3 % y una pérdida de 0.39, lo cual representa un rendimiento bueno considerando que se entrenó con una cantidad moderada de datos en relación con la complejidad del modelo RoBERTa. Este desempeño se ve reforzado por la transferencia de conocimiento al uso de un modelo preentrenado. No obstante, las curvas de entrenamiento evidencian un patrón de sobreajuste. A nivel de métricas por clase, se observa un desequilibrio moderado: el modelo obtiene un F1-score de 0.754 para la clase 0 y 0.885 para la clase 1. Aunque el rendimiento global es sólido, esta brecha sugiere que el modelo sigue favoreciendo a la clase mayoritaria.

Por su parte, el modelo balanceado, entrenada con 500.000 ejemplos, alcanzó los mejores resultados en la tarea binaria, con una precisión del 91,7 % y una pérdida de 0.22, convirtiéndose en el modelo con el rendimiento más alto entre todas las arquitecturas evaluadas. Cabe destacar que este desempeño fue logrado en tan solo dos épocas de entrenamiento, debido a restricciones computacionales. Si bien las gráficas de entrenamiento sugieren una leve tendencia al sobreajuste, esta diferencia es mínima. Además, la tabla de métricas por clase evidencia una excelente consistencia y equilibrio: el F1-score fue de 0.906 para la clase 0 y 0.926 para la clase 1. Esta distribución equitativa, junto con una precisión y recall superiores al 90 % en ambas clases, refuerza la conclusión de que un conjunto balanceado, incluso con pocas épocas, permite a RoBERTa alcanzar un buen resultado en tareas de clasificación binaria.

Finalmente, el modelo ponderado obtuvo una precisión del 84 %, con una pérdida ligeramente inferior a la del modelo sin balanceo (0.36), lo que indica un rendimiento muy similar entre ambas configuraciones. Sin embargo, al observar las curvas de entrenamiento, se aprecia nuevamente un patrón de sobreajuste, posiblemente por la limitada cantidad de datos disponibles para un modelo tan complejo como RoBERTa. Lo más llamativo es que, a pesar de aplicar penalizaciones a las clases mayoritarias, persiste un desequilibrio entre clases, como se evidencia en los valores de F1-score. Aunque el desempeño general es aceptable, con un F1-score de 0.814, estos resultados reflejan que el modelo sigue clasificando mejor la clase predominante, lo que es decepcionante considerando que la ponderación buscaba precisamente corregir este sesgo. El modelo ponderado no logró superar al sin balanceo, y en algunos aspectos parece incluso ligeramente inferior, posiblemente por una mala calibración de los pesos o porque el modelo no los integró adecuadamente durante el entrenamiento.

Multiclase

En el caso multiclase, el modelo Transformer fue evaluado también bajo tres configuraciones: sin balanceo, con balanceo estricto y con ponderación de clases.

En la configuración sin balanceo, el modelo alcanzó una precisión del 52,7 % y una pérdida de 1.02, superando al resto de configuraciones multiclase. Este rendimiento, aunque modesto, fue el más alto entre los tres enfoques evaluados, lo cual se explica por el uso de la totalidad del conjunto de datos durante el entrenamiento. No obstante, las gráficas de entrenamiento muestran un claro sobreajuste. Además, la matriz de confusión y las métricas por clase evidencian un sesgo claro hacia las clases mayoritarias, que presentan los F1-scores más altos. En contraste, clases como la 1 y la 2 obtienen F1-scores bajos, lo que indica que el modelo tiene dificultades para distinguir adecuadamente entre las clases intermedias o menos representadas.

Por su parte, el modelo balanceado alcanzó una precisión global de 44,9 % y una pérdida de 1.17, siendo el que presentó el rendimiento más bajo entre las configuraciones evaluadas. Esta caída se atribuye a la reducción del volumen de datos. Además, las curvas de entrenamiento evidencian un sobreajuste temprano. Sin embargo, al analizar la matriz de confusión y las métricas por clase, se observa que el modelo intenta formar una diagonal coherente, señal de que está captando correctamente la lógica de clasificación, aunque con errores frecuentes. Por ejemplo, la clase 4 alcanza el mayor recall , lo que indica que el modelo la predice con frecuencia, pero su precisión es baja, reflejando muchas predicciones incorrectas hacia esta clase.

Finalmente, el modelo ponderado alcanzó una precisión del 46,9 % y una pérdida de 1.14, quedando en un punto intermedio entre los modelos sin balanceo y balanceado. Aunque utilizó todo el conjunto de datos y aplicó pesos para reducir el impacto del desbalance, los resultados muestran que estas ponderaciones no fueron efectivas. Además, las curvas de entrenamiento evidencian sobreajuste, posiblemente acentuado por la complejidad del modelo y la escasez de datos. La tabla de métricas muestra que el modelo sigue sesgado hacia la clase 4 , que tiene el F1-score más alto (0.641) y un recall de 0.74, seguida de la clase 0 con un F1-score de 0.541. La clase 1 es la más problemática, con un F1-score de apenas 0.321 y precisión de 0.267.

General

En términos generales, el modelo Transformer demostró una capacidad destacada para resolver la tarea de clasificación binaria, alcanzando altos niveles de precisión y consistencia cuando se disponía de un conjunto balanceado de datos. Este caso evidenció que, con datos bien distribuidos, incluso un número reducido de épocas es suficiente para lograr un rendimiento sobresaliente, con métricas por clase equilibradas y un F1-score macro superior al 0.91. En contraste, cuando no se aplicaron técnicas de balanceo o se utilizaron ponderaciones para compensar el desbalance, los resultados fueron aceptables, pero con un sesgo evidente hacia la clase mayoritaria. Esto sugiere que, aunque el modelo aprovecha bien el conocimiento preentrenado, su capacidad de generalización se ve limitada por la distribución de los datos, siendo especialmente sensible a los desequilibrios.

En el escenario multiclase, el comportamiento fue más irregular. La configuración sin balanceo, pese a mostrar la mejor precisión global, sufrió un sobreajuste y un marcado sesgo hacia las clases de mayor frecuencia, especialmente la de 5 estrellas. El modelo balanceado, por su parte, presentó el rendimiento más bajo en exactitud , pero mostró señales de una clasificación más equitativa, aunque con problemas de generalización por la

reducción del conjunto de entrenamiento. El enfoque ponderado, que buscaba un punto medio, tampoco logró mitigar el sesgo, manteniendo una tendencia similar a la configuración sin balanceo y con un desempeño sólo ligeramente superior al modelo balanceado.

Conclusiones

En conclusión, la evaluación de múltiples modelos de clasificación de reseñas de Yelp permitió identificar diferencias claras en su desempeño bajo distintas condiciones de entrenamiento. En general, la tarea binaria resultó significativamente más manejable que la clasificación multiclase. Todos los modelos obtuvieron precisiones mucho mayores en binario que en la tarea de cinco clases, donde las precisiones fueron más modestas. Esta brecha de rendimiento es esperable dado que el problema binario simplifica la predicción y mitiga parcialmente el desequilibrio de clases al agrupar las reseñas en solo dos categorías.

El modelo base MLP sirvió como línea de referencia y mostró limitaciones importantes de generalización. Aunque pudo aprender el conjunto de entrenamiento casi a la perfección, presentó una fuerte tendencia al sobreajuste, fallando en reproducir ese rendimiento en validación o prueba. Esto se atribuye a su incapacidad para capturar relaciones contextuales en texto secuencial: el MLP tiende a memorizar patrones específicos de los datos de entrenamiento sin extraer características generales. En consecuencia, su desempeño en clasificación de reseñas fue el más bajo del grupo, especialmente en el escenario multiclase, donde quedó rezagado frente a modelos secuenciales.

Las arquitecturas recurrentes introdujeron mejoras notables. El RNN simple logró un rendimiento superior al MLP al incorporar memoria temporal, pero evidenció problemas al manejar secuencias largas y datos desequilibrados. En la clasificación multiclase sin balanceo, por ejemplo, el RNN alcanzó la mayor precisión global entre los enfoques básicos, pero su matriz de confusión tuvo un sesgo severo. Por otro lado, las curvas de aprendizaje del RNN mostraron generalmente inestabilidad, señal de dificultades durante el entrenamiento quizá relacionadas con la arquitectura limitada para secuencias extensas.

El modelo LSTM demostró un comportamiento superior y más estable que el RNN. Gracias a su mecanismo de memoria de largo plazo, la LSTM logró capturar dependencias contextuales más profundas, traduciéndose en mejores métricas tanto en binario como en multiclase. En los experimentos multiclase con el dataset original, la LSTM sin técnicas de balanceo alcanzó ~43% de exactitud, mostrando una capacidad mejorada para diferenciar clases de reseña. Asimismo, bajo condiciones balanceadas o con ponderación de clases, las LSTM mantuvieron un desempeño más alto que las RNN. Sin embargo, con datos limitados también enfrentaron sobreajuste.

Finalmente, el modelo Transformer basado en RoBERTa preentrenado sobresalió con el mejor rendimiento global entre todas las arquitecturas probadas. En clasificación binaria con datos balanceados, el Transformer alcanzó la precisión más alta junto con métricas por clase muy equilibradas. Incluso entrenado por pocas épocas, este modelo aprovechó el conocimiento previo de RoBERTa para lograr predicciones precisas, superando a los demás modelos. En escenarios binarios sin balanceo o con ponderación, el Transformer también mostró desempeños sólidos, aunque con indicios de sesgo hacia la clase mayoritaria si no se equilibraban los datos. En la tarea multiclase, el Transformer logró la mejor exactitud con el conjunto original, lo que confirma su superioridad incluso con datos moderados. No obstante, persistieron dificultades para distinguir clases intermedias. Aplicar balanceo estricto en multiclase redujo este sesgo pero llevó su exactitud a disminuir, debido a la disminución de datos de entrenamiento.

Los experimentos confirmaron que el desbalance de clases influye en el comportamiento de los modelos. Entrenando con el conjunto original, se obtuvo frecuentemente la mayor exactitud global en ambas tareas, pero a costa de injusticia entre clases: todos los modelos tendieron a sobre-clasificar las categorías mayoritarias. Esto se evidenció en matrices de confusión sesgadas. Por el contrario, el balanceo estricto mediante submuestreo aseguró que cada clase estuviera representada equitativamente en el entrenamiento, logrando efectivamente mejorar la equidad. Sin embargo, esta estrategia sacrificó precisión global, especialmente en la clasificación multiclase. Al reducir el volumen de datos, los modelos tuvieron dificultad para aprender patrones generales, sufriendo mayor varianza e incluso comportamientos cercanos al azar en algunos casos extremos.

La ponderación de clases se evaluó como una estrategia para mitigar el desbalance sin recurrir a la reducción del conjunto de datos original, asignando mayores penalizaciones a los errores cometidos en clases minoritarias. Sin embargo, en términos generales, esta técnica no logró cumplir su propósito principal, reducir el sesgo hacia las clases mayoritarias. En ninguno de los modelos evaluados, la ponderación consiguió eliminar el sesgo, y en algunos casos incluso lo acentuó. Aunque en ciertas configuraciones se observó una ligera mejora en la exactitud global o una mayor estabilidad en las curvas de entrenamiento, estos beneficios fueron marginales frente al hecho de que el modelo continuó clasificando de forma desbalanceada.

Otra oclusión de los experimentos fue que, a mayor volumen de datos de entrenamiento, mejoraron significativamente el desempeño y la estabilidad de los modelos, efecto especialmente notable al combinarse con técnicas de balanceo de clases. Al escalar el conjunto de reseñas desde miles hasta varios millones de ejemplos, los modelos incrementaron no sólo su exactitud, sino también su capacidad para mitigar el sobreajuste y distribuir de manera más equitativa sus predicciones entre las distintas clases. Aunque persistieron ciertos errores en clases intermedias, el notable aumento de la exactitud evidenció que los modelos lograron aprender patrones más representativos cuando dispusieron de suficiente volumen y diversidad por categoría, demostrando que con datasets masivos la penalización habitual por balanceo deja de ser una limitante. Cuando esto no sea posible, simplificar la tarea mediante la agrupación de clases o su reformulación como un problema binario podría ser una estrategia efectiva para mejorar sustancialmente el rendimiento general del modelo.

En resumen, este estudio comparativo mostró que las arquitecturas más sofisticadas y los conjuntos de datos balanceados y abundantes producen los mejores resultados en clasificación de texto.

Bibliografía

- Keras Team. (s. f.). *TextVectorization layer*. En *Keras API*. Recuperado el 22 de julio de 2025, de https://keras.io/api/layers/preprocessing_layers/text/text_vectorization/
- TensorFlow Core. (2023). *Text classification with an RNN*. En *TensorFlow Text Tutorials*. Recuperado el 22 de julio de 2025, de https://www.tensorflow.org/text/tutorials/text_classification_rnn?hl=es-419
- TensorFlow Developers. (s. f.). *Working with RNNs in Keras*. En *TensorFlow Guide*. Recuperado el 22 de julio de 2025, de https://www.tensorflow.org/guide/keras/working_with_rnns
- Keras Team. (s. f.). *LSTM layer*. En *Keras API*. Recuperado el 22 de julio de 2025, de https://keras.io/api/layers/recurrent_layers/lstm/
- Keras Team. (s. f.). *Text classification with Transformer*. En *Keras examples: NLP*. Recuperado el 22 de julio de 2025, de https://keras.io/examples/nlp/text_classification_with_transformer/
- Keras Team. (s. f.). *Roberta Text Classifier*. En *KerasHub: Pretrained Models*. Recuperado el 22 de julio de 2025, de https://keras.io/keras_hub/api/models/roberta/roberta_text_classifier/
- Invernizzi, L., Long, J., Chollet, F., O'Malley, T., & Jin, H. (2019, 31 de mayo; última modificación 27 de octubre de 2021). *Getting started with KerasTuner*. Recuperado el 22 de julio de 2025, de https://keras.io/keras_tuner/ keras.io+5keras.io+5keras.io+5